



**SAKARYA  
UYGULAMALI BİLİMLER  
ÜNİVERSİTESİ**

**AD : SEDANUR**

**SOYAD : PEKER**

**ÖĞRENCİ NO : 22010903060**

**ÖDEVİN KONUSU : PİCOBLAZE MİKROİŞLEMCİSİ**

**DERSİN ÖĞRETMENİ : HALİT ÖZTEKİN**

## Giriş

Mikroişlemci, bir elektronik cihazın "beyni" olarak tanımlanabilecek, çok amaçlı, programlanabilir ve saatle senkronize çalışan bir bileşendir. Bellekten aldığı ikili (binary) talimatları işleyerek sonuçları çıktı olarak sunar. Bu cihaz, kendisine verilen talimatlara göre karmaşık işlemleri kısa sürede tamamlayabilir.

Bir mikroişlemci, temel olarak veri depolamak için registerlar, aritmetik ve mantıksal işlemleri gerçekleştirmek için ALU, komutları çözümlemek için kod çözücüler, işlem akışını kontrol etmek için kontrol hatları gibi bileşenler içerir. Mikroişlemcinin işleyişi, getirme (fetch), kod çözme (decode) ve yürütme (execute) döngüsü üzerine kuruludur. Mikroişlemci, bir "durdurma" talimatına kadar bu döngüyü tekrar ederek işlem yapar.

Günümüzde mikroişlemciler, farklı mimariler ve uygulama alanlarına uygun olarak çeşitlenmiştir. Örneğin, 8085 mikroişlemcisi, temel mikroişlemci kavramlarını öğrenmek için uygun bir yapı sunarken; PicoBlaze gibi modern soft işlemciler, gömülü sistemler ve FPGA (Field Programmable Gate Array) tabanlı tasarımlarda esneklik ve verimlilik sağlar. FPGA, donanım seviyesinde yeniden programlanabilir bir entegre devre olarak dikkat çeker.

Bu rapor, 8085 ve PicoBlaze mikroişlemcilerinin mimari ve işlevsel farklılıklarını incelemekte ve bu iki işlemcinin modern mühendislik uygulamalarındaki rollerini karşılaştırmaktadır. Ayrıca PicoBlaze işlemcisi üzerine bir makale özetlenmiş ve bir uygulama örneği geliştirilmiştir. Bu çalışma, mikroişlemci teknolojisinin temel yapı taşlarından modern çözümlere doğru evrimini ele almayı amaçlamaktadır.

## PicoBlaze Mikroişlemcisi

### Genel Amaçlı Kayıtlar

PicoBlaze mikrodenetleyicisi, s0 ile sF arasında adlandırılmış 16 baytlık genel amaçlı kayıtlar sunar. Bu kayıtlar, işlemci tarafından esnek bir şekilde kullanılabilir ve programın netliğini artırmak amacıyla assembler yönergeleriyle yeniden adlandırılabilir. Bu tasarım, mikrodenetleyicinin daha verimli kullanılmasını sağlar, çünkü her bir kayıt, programın ihtiyaçlarına göre yeniden işlevlendirilebilir. Hiçbir kayıt, özel bir görev için ayrılmamış ya da diğerlerine göre öncelikli değildir. Yani, her kayıt birbirinin yerine geçebilecek şekilde tasarlanmış olup, işlemcinin herhangi bir kaydı herhangi bir durumda kullanabilmesini sağlar. Ayrıca, PicoBlaze tasarımında geleneksel bir akümülatör bulunmaz; tüm hesaplamalar ve veri işleme işlemleri, doğrudan bu genel amaçlı kayıtlarda yapılır ve sonuçlar bu kayıtlarda saklanır. Bu yaklaşım, programın daha esnek ve daha hızlı çalışmasına olanak tanır.

### 1.024 KB'lık Talimat Programı Deposu

PicoBlaze mikrodenetleyicisi, FPGA içinde 1.024 KB'lık talimatlık bir program deposu kapasitesine sahip olup, her bir talimat 18 bit genişliğinde olacak şekilde yapılandırılmıştır. Bu talimatlar, FPGA tasarımı sırasında derlenir ve otomatik olarak yüklenir. Bu esneklik, tasarımcıların FPGA üzerinde etkileşimli kod güncellemeleri yapmalarına veya aynı FPGA üzerinde birden fazla PicoBlaze mikrodenetleyicisi kullanmalarına olanak tanır. Bu sayede, FPGA tasarımını yeniden derlemeye gerek kalmadan yazılımda yapılan değişiklikler anında uygulanabilir. Bu, özellikle gelişmiş sistemlerde esneklik sağlayan bir özelliktir, çünkü işlemci üzerinde yapılan her türlü değişiklik, FPGA'nın derlenmesine gerek kalmadan hızla test edilebilir.

## Aritmetik ve Mantık Birimi (ALU)

PicoBlaze mikrodnetleyicisinin Aritmetik ve Mantık Birimi (ALU), bayt genişliğinde çeşitli hesaplama işlemlerini gerçekleştirebilecek şekilde tasarlanmıştır. ALU, temel aritmetik işlemlerden mantık işlemlerine kadar geniş bir işlem yelpazesi sunar. Bu işlemler arasında:

- **Aritmetik işlemler:** Toplama ve çıkarma gibi temel matematiksel hesaplamalar,
- **Mantık işlemleri:** AND, OR, XOR gibi bit düzeyindeki işlemler,
- **Karşılaştırma işlemleri:** Aritmetik karşılaştırmalar ve bit düzeyindeki testler,
- **Kaydırma ve döndürme işlemleri:** Veri manipülasyonu için gerekli olan kaydırma ve döndürme işlemleri.

ALU, bu işlemleri mikrodnetleyicisinin belirli bir kaydında (sX) gerçekleştirir ve işlemin sonucu aynı kayıta saklanır. Eğer bir işlemde ikinci bir işlenen gereksinimi varsa, bu işlenen başka bir kayıt (sY) veya 8 bitlik bir sabit değer (kk) olabilir. Bu esnek yapı, mikrodnetleyicisinin işlem gücünü artırır ve daha karmaşık hesaplamaların yapılmasına imkan verir.

## Bayraklar

ALU işlemleri sırasında çeşitli bayraklar etkinleşir ve bu bayraklar, yapılan işlemin sonucuyla ilgili önemli bilgiler sağlar. Özellikle, ZERO ve CARRY bayrakları önemlidir. ZERO bayrağı, bir işlem sonucu sıfır elde edildiğinde set edilir, yani işlemin sonucu hiçbir değişiklik yapmamıştır. CARRY bayrağı ise işlemin taşıma durumu hakkında bilgi verir ve genellikle aritmetik işlemlerin sonucunda ortaya çıkar. Ek olarak, INTERRUPT\_ENABLE bayrağı, kesme işlemleri için kullanılan bir bayraktır ve bu bayrak etkinleştirildiğinde, mikrodnetleyici harici olaylara kesme ile yanıt verebilir. Bu bayraklar, mikrodnetleyicisinin doğru bir şekilde çalışmasını sağlamak ve çeşitli işlemler için gerekli bilgileri kullanıcıya sunmak adına kritik öneme sahiptir.

## 64 Baytlık Scratchpad RAM 6 Bit

PicoBlaze mikrodnetleyicisi, verimli ve düşük maliyetli bir sistem tasarımı için 64 baytlık bir dahili Scratchpad RAM sunar. Bu RAM, veri işlemlerinde büyük kolaylık sağlar çünkü mikrodnetleyiciye doğrudan erişim imkanı verir. Bu bellek, iki temel komut olan STORE ve FETCH ile adreslenebilir. STORE komutu, verileri 16 farklı kayıttan birine yazarken, FETCH komutu ise RAM'den veri okuma işlemi yapar ve bu veriyi yine kayıtlara aktarır.

Scratchpad RAM'in kullanımı, mikrodnetleyicisinin işlem gücünü artıran bir özellik sağlar. 6 bitlik adresleme ile yalnızca 00-3F hexadecimal aralığında veri depolanabilir. Bu sınırlı alan, işlemciye daha hızlı veri erişimi sağlar, çünkü veri işlemleri doğrudan RAM içinde yapılır ve daha hızlı bir işlem gerçekleştirilir. Ayrıca, giriş/çıkış (G/Ç) işlemleri için ayrı bir alan ayırabilme imkanı sunarak, işlemcinin veri yönetimindeki esnekliğini artırır. Bu özellik, düşük maliyetli ve yüksek verimli sistem tasarımlarında büyük bir avantaj sağlar.

## Giriş/Çıkış (G/Ç) Birimleri

PicoBlaze mikrodnetleyicisi, 256 giriş ve 256 çıkış bağlantı noktası sunarak geniş bir G/Ç yelpazesi sağlar. Bu yapı, mikrodnetleyicisinin çevre birimleriyle ya da diğer FPGA mantık birimleriyle esnek bir şekilde etkileşim kurmasını mümkün kılar. G/Ç bağlantı noktaları, işlemcinin dış dünya ile olan iletişimini düzenler. PORT\_ID çıkışı, veri iletimi veya alımı yapılacak olan bağlantı noktasının adresini belirler. Giriş işlemlerinde, IN\_PORT üzerinden alınan veri, mikrodnetleyicisinin belirli bir kaydına

yazılırken; çıkış işlemlerinde, bir kaydın içeriği OUT\_PORT bağlantı noktasına iletilir.

Bu yapı, çevresel cihazlarla etkileşimde büyük bir esneklik sağlar ve işlemcinin performansını artıran bir özellik sunar. Çevre birimleri ile verimli bir veri akışı sağlayarak, zamanlama gerektiren işlemlerde hızlı yanıt alınmasına imkan tanır. FPGA mantığı ile entegrasyonu sayesinde, sistemin daha yüksek performansta çalışmasını ve çevre birimlerinin etkin bir şekilde kullanılmasını sağlar.

### **Program Sayacı (PC) 10 bit**

PicoBlaze mikrodenetleyicisinin Program Sayacı (PC), programın yürütülecek bir sonraki talimatının adresini belirler. PC'nin çalışması, işlemcinin doğru bir şekilde talimatları takip etmesini sağlar. Her talimat yürütüldükten sonra, PC bir sonraki talimatın adresine geçer. Ancak, program akışını değiştirmek gerektiğinde, JUMP, CALL, RETURN gibi komutlar kullanılabilir. Bu komutlar, programın akışını yönlendirir ve işlemin doğru sırayla yapılmasını sağlar.

PicoBlaze'in PC'si 10 bitlik bir değere sahiptir ve bu sayede 1.024 talimatlık bir kod alanını destekler. Bu uzunluk, programın veri alanını belirler ve 3FF hexadecimal adresine ulaşıldığında, PC yeniden başlangıç adresine döner. Ayrıca, PC'nin doğrudan uygulama kodu tarafından değiştirilmesi mümkün değildir, bu da güvenli ve hatasız bir çalışma sağlar. Programın sırasının düzenli bir şekilde takip edilmesi, hata payını minimuma indirir ve mikrodenetleyicinin daha stabil çalışmasına yardımcı olur.

### **Program Akış Kontrolü**

PicoBlaze mikrodenetleyicisi, program akışını kontrol etmek için oldukça esnek bir yapı sunar. Program akışını değiştirebilmek, özellikle karmaşık uygulamalarda büyük fayda sağlar. JUMP komutları, programın herhangi bir adresine atlama yapılmasını sağlarken, CALL ve RETURN komutları alt yordamlar için kullanılır. Bu alt yordamlar, kodun tekrar kullanımını sağlar ve yazılımın daha modüler olmasına imkan tanır. CALL komutu, bir alt yordamın başlama adresini belirtirken, dönüş adresi otomatik olarak CALL/RETURN yığınının saklanır.

Kesme (interrupt) işlemleri aktif olduğunda, bir kesme olayı meydana geldiğinde, PC kesme vektörüne yönlendirilir ve kesme hizmet rutini tamamlandığında program normal akışına geri döner. Bu tür akış kontrolü, zamanlama kritik sistemlerde önemli bir avantaj sağlar çünkü herhangi bir kesinti durumunda program hızlıca kesme rutinine geçebilir ve ardından sorunsuz bir şekilde işlemeye devam eder. PicoBlaze'in program akış kontrolü, özellikle gerçek zamanlı uygulamalarda büyük bir esneklik ve yüksek performans sunar.

### **CALL/RETURN Yığını**

PicoBlaze mikrodenetleyicisi, 31 düzeye kadar iç içe çağrılabilen (CALL) komutları destekleyen özel bir donanım yığına sahiptir. Bu donanım yığını, sadece işlevsel komutların adreslerini depolamakla kalmaz, aynı zamanda kesme işlemleri sırasında da aktif olarak kullanılır. Kesme işlemleri etkinleştirildiğinde, yığının bir kısmı, bu kesme taleplerini işlemek amacıyla ayrılmalıdır. Bu özellik, yığın belleği kullanımını minimize ederek sistemin daha verimli çalışmasını sağlar. Yığın, döngüsel bir tampon yapısı olarak tasarlanmıştır ve kapasitesi dolduğunda en eski veriyi silerek yenilerini ekler. İlginç bir şekilde, yığın kontrolü için herhangi bir özel komut bulunmaması, tasarımı daha esnek ve basit kılar. Ayrıca, yığın, program belleğini kullanmadığı için daha az sistem kaynağı tüketir, bu da genel kaynak verimliliğini artırır ve tasarımın daha kompakt olmasına katkı sağlar.

## **Kesintiler (Interrupts)**

PicoBlaze mikrodenetleyicisi, harici olaylara hızlı bir şekilde tepki verebilmek için bir INTERRUPT girişine sahiptir. "Asenkron" olarak tanımlanan bu özellik, kesme olaylarının komut döngüsünün herhangi bir anında tetiklenebilmesini sağlar. Ancak, sistemin kararlılığını artırmak amacıyla, tüm girişlerin PicoBlaze denetleyicisiyle senkronize edilmesi önerilir. Bu yaklaşım, sistemin daha stabil ve güvenilir çalışmasını sağlar. PicoBlaze mikrodenetleyicisi, kesmelere yalnızca beş saat döngüsü içinde tepki verir, bu da gerçek zamanlı uygulamalar için büyük bir avantaj sağlar. Yüksek tepki süresi, FPGA tabanlı projelerde kesintisiz ve hızlı yanıtlar alınmasını mümkün kılar, böylece uygulama daha hassas ve hızlı çalışabilir.

## **Neden PicoBlaze Mikrodenetleyicisi?**

PicoBlaze mikrodenetleyicisi, özellikle Xilinx FPGA'larıyla uyumlu olacak şekilde geliştirilmiş bir mimariye sahiptir ve Spartan-3, Spartan-6 ve Virtex-6 gibi farklı FPGA ailelerini destekler. Bu mikrodenetleyicinin kompakt yapısı, aynı performansı sağlarken daha az FPGA kaynağı kullanır, bu da daha verimli bir sistem tasarımı anlamına gelir. PicoBlaze'ın düşük kaynak tüketimi, daha düşük maliyetli çözümler sunarak tasarım sürecini daha verimli hale getirir.

Ayrıca, PicoBlaze, Xilinx FPGA'larında telifsiz bir şekilde yeniden kullanılabilir ve açık kaynak kodu (VHDL formatında) sunulur. Bu, ürünün zamanla eskime riskini ortadan kaldırır ve gelecekteki geliştirmeler için uyum sağlamasını mümkün kılar. Mikrodenetleyici ayrıca genişletilebilir ve özelleştirilebilir, bu da tasarımcıların ihtiyaçlarına göre esneklik sunar.

PicoBlaze, MicroBlaze gibi daha büyük işlemcilerin geliştirilmesinden önce, mikrodenetleyicilerin genellikle FPGA dışında bulunan bileşenler olduğu bir dönemde, FPGA içinde doğrudan entegre edilerek çok daha yüksek bir performans elde edilmesini sağlar. FPGA kaynaklarıyla doğrudan bağlantı kurarak, daha verimli ve hızlı uygulamalar mümkün hale gelir. Bu sayede, FPGA dışındaki cihazlarla olan bağlantı sınırları ortadan kaldırılır.

Bir başka avantaj ise, sistem maliyetlerini düşürmesidir. PicoBlaze, FPGA içinde kullanılmayan kaynakları işgal eder ve bazen birden fazla PicoBlaze mikrodenetleyicisi paralel çalışarak farklı görevleri yerine getirebilir. Bu yaklaşım, karmaşık sistemlerin daha verimli ve düşük maliyetle tasarlanmasına olanak tanır.

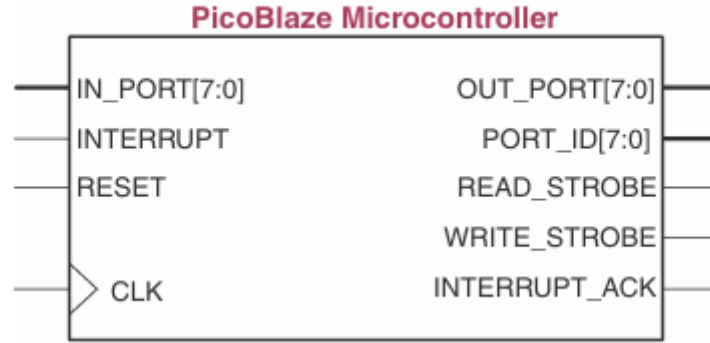
## **Neden FPGA İçinde Mikrodenetleyici Kullanılmalı?**

Mikrodenetleyiciler ve FPGA'lar, dijital mantık işlevlerini yerine getirme açısından farklı avantajlar sunar. Mikrodenetleyiciler, genellikle genel kullanım amaçlı ve kontrol odaklı uygulamalar için ideal cihazlardır. Sabit program belleği gereksinimlerine sahip daha basit projelerde, mikrodenetleyiciler oldukça etkili çözümler sunar. Ancak, FPGA içinde entegre edilmiş bir mikrodenetleyici, daha esnek ve optimize edilmiş tasarımlar ortaya koyar. FPGA, paralel işlem yeteneği sunduğundan, mikrodenetleyicinin sınırlı performansını aşabilir. Mikrodenetleyiciler genellikle sıralı bir işlem yapısına sahip olduğundan, karmaşık görevlerde işlem süresi uzayabilir.

Bu noktada, FPGA'nın paralel işleme kapasitesi devreye girer. FPGA, birden fazla işlemi aynı anda gerçekleştirebilir ve nanosaniye ölçeğinde hızlı yanıt verebilir. Bu, zamanlama açısından kritik olan uygulamalarda önemli bir avantaj sağlar. FPGA içinde entegre edilmiş bir mikrodenetleyici, her iki teknolojinin güçlü yönlerini birleştirerek daha verimli ve güçlü bir çözüm sunar. Mikrodenetleyici, kontrol ve zamanlama gibi esnek işlevleri üstlenirken, FPGA mantığı yüksek performans gerektiren paralel işlemleri gerçekleştirir. Bu işbirliği, karmaşık projelerde hem esneklik hem de performans optimizasyonu sağlar.

## PicoBlaze Arayüz Sinyalleri

PicoBlaze mikrodnetleyicisinin arayüz sinyalleri, cihazın çevre birimleriyle ve diğer sistemlerle etkileşime geçebilmesi için önemli bağlantıları ve işlevsellikleri sağlar. Bu sinyaller aşağıdaki şekilde gösterilmiştir.



|               |        |                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|---------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IN_PORT[7:0]  | Input  | Bu sinyal, bir GİRİŞ komutu çalıştırıldığında, PicoBlaze mikrodnetleyicisinin giriş verilerini aldığı bağlantı noktasıdır. Veriler, sistem saatinin (CLK) yükselen kenarlarında (yani her bir saat döngüsünün başında) alınır ve işlenir. Bu, cihazın dış dünyadan veri almasını sağlayan önemli bir bağlantıdır.                                                                                                                  |
| INTERRUPT     | Input  | Eğer INTERRUPT_ENABLE bayrağı uygulama kodu tarafından etkinleştirilmişse, bu giriş üzerinden bir kesme (interrupt) olayı oluşturulabilir. Bir kesme olayı, en az iki saat döngüsü boyunca bu giriş sinyalinin yüksek (high) olması ile tetiklenir. Eğer INTERRUPT_ENABLE bayrağı temizlenirse, bu giriş sinyali yok sayılır ve kesme olayı gerçekleşmez. Bu sinyal, dış olaylara hızlı bir şekilde tepki vermek için kullanılır.  |
| RESET         | Input  | PicoBlaze mikrodnetleyicisini sıfırlamak için kullanılan bu sinyal, RESET olayını başlatmak amacıyla bir saat döngüsü boyunca yüksek olmalıdır. FPGA yapılandırması tamamlandıktan sonra, otomatik olarak bir sıfırlama olayı tetiklenir. Bu sinyal, sistemin başlangıç durumuna dönmelerini sağlar.                                                                                                                               |
| CLK           | Input  | PicoBlaze mikrodnetleyicisi, bu saat sinyaline göre çalışır. Saat sinyali, belirli bir frekansta (Xilinx ISE® yazılımı tarafından belirtilen maksimum çalışma frekansı kadar) akar. PicoBlaze içindeki tüm senkronize elemanlar, saat döngüsünün yükselen kenarından itibaren işlevlerini yerine getirir. Burada önemli olan, FPGA'nın minimum darbe genişliği gereksinimlerinin ötesinde saat döngüsü gereksinimi bulunmamasıdır. |
| OUT_PORT[7:0] | Output | Çıkış verileri, ÇIKIŞ komutu verildiğinde bu bağlantı noktasına yazılır. Veriler, iki saat döngüsü boyunca bu portta görünür. Bu veriler, CLK'nın yükselen kenarında yakalanarak FPGA içindeki çıkış verisini olarak işlenir. WRITE_STROBE sinyali, çıkış verilerinin doğruluğunu onaylamak için kullanılır.                                                                                                                       |
| PORT_ID[7:0]  | Output | Giriş ve çıkış komutları çalıştırılırken, PORT_ID bağlantı noktasının adresini belirtir. Bu sinyal, giriş veya çıkış komutları sırasında iki saat döngüsü boyunca ilgili port adresini gösterir.                                                                                                                                                                                                                                   |
| READ_STROBE   | Output | READ_STROBE sinyali, yüksek olarak ayarlandığında, PicoBlaze mikrodnetleyicisi IN_PORT[7:0] bağlantı noktasındaki giriş verilerinin GİRİŞ komutu ile doğru kayda yazıldığını belirtir. Bu sinyal, iki döngülü GİRİŞ komutunun ikinci saat döngüsünde etkinleştirilir ve genellikle FIFO (First-In-First-Out) bellek yapılarından okuma işlemleri için kullanılır.                                                                  |
| WRITE_STROBE  | Output | WRITE_STROBE sinyali, yüksek olduğunda, çıkış verilerinin ÇIKIŞ komutu sırasında doğru şekilde çıkış portuna                                                                                                                                                                                                                                                                                                                       |

|               |        |                                                                                                                                                                                                                                                                                                              |
|---------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               |        | yazıldığını doğrular. Bu sinyal, iki döngülü ÇIKIŞ komutunun ikinci saat döngüsünde aktive edilir ve FPGA içindeki çıkış verilerini doğrular.                                                                                                                                                                |
| INTERRUPT_ACK | Output | Bu sinyal, bir kesme olayı meydana geldiğinde yüksek olarak ayarlanır ve bu olayın kabul edildiğini belirtir. INTERRUPT_ACK sinyali, iki döngülü bir KESME olayının ikinci saat döngüsünde etkinleştirilir. Ayrıca, bu sinyal, kesme girişinin kaynağını temizlemek için isteğe bağlı olarak kullanılabilir. |

Picoblaze'in komut setleri rapor sonunda verilmiştir. Bu komutlar 5 grupta inceleyebilir:

| Adres Alanı                      | Boyut<br>(Derinlik x Genişlik) | Adresleme Modları | Adres Alanında Çalışan Yönergeler                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------------|--------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Talimat                          | 1Kx18                          | Doğrudan          | <ul style="list-style-type: none"> <li>• JUMP</li> <li>• CALL</li> <li>• RETURN</li> <li>• RETURNI</li> <li>• INTERRUPT</li> <li>• RESET</li> </ul> Diğerleri bilgisayarı bir sonraki konuma yükseltir                                                                                                                                                                                                                                                                           |
| Register File<br>(Kayıt Dosyası) | 16x8                           | Doğrudan          | <ul style="list-style-type: none"> <li>• LOAD</li> <li>• AND</li> <li>• OR</li> <li>• XOR</li> <li>• TEST (read only)</li> <li>• ADD</li> <li>• ADDCY</li> <li>• SUB</li> <li>• SUBCY</li> <li>• COMPARE (read only)</li> <li>• SR0</li> <li>• SR1</li> <li>• SRX</li> <li>• SRA</li> <li>• RR</li> <li>• SL0</li> <li>• SL1</li> <li>• SLX</li> <li>• SLA</li> <li>• RL</li> <li>• INPUT</li> <li>• OUTPUT (read only)</li> <li>• STORE (read only)</li> <li>• FETCH</li> </ul> |
| Scratchpad RAM                   | 64x8                           | Doğrudan, Dolaylı | <ul style="list-style-type: none"> <li>• STORE</li> <li>• FETCH</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                       |
| I/O                              | 256x8                          | Doğrudan, Dolaylı | <ul style="list-style-type: none"> <li>• INPUT</li> <li>• OUTPUT</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                      |
| CALL/RETURN Stack                | 31x10                          | N/A (YOK)         | <ul style="list-style-type: none"> <li>• CALL</li> <li>• Enabled INTERRUPT</li> <li>• RETURN</li> <li>• RETURNI</li> <li>• RESET</li> </ul>                                                                                                                                                                                                                                                                                                                                      |

## 1. Processing Data (Verilerin İşlenmesi)

PicoBlaze mikrodenetleyicisi, veri işleme işlemlerini 16 genel amaçlı kayıt üzerinden gerçekleştirir. Bu kayıtlar, işlenen veriyi depolamak için kullanılır. Ancak, yalnızca veri işleme talimatları, özellikle ZERO ve CARRY bayraklarını değiştirebilir. Bu tür işlemler dört ana gruba ayrılır:

- a) Mantık Talimatları
- b) Aritmetik Talimatlar
- c) Test ve Karşılaştırma Talimatları
- d) Kaydırma ve Döndürme Talimatları

### a) Mantık Talimatları

Mantık talimatları, iki işlenen arasındaki bit düzeyinde mantıksal işlemleri gerçekleştiren talimatlardır. Bu işlemler, genellikle AND, OR ve XOR gibi temel mantıksal işlemleri içerir. İlk işlenen genellikle bir kayıt olurken, ikinci işlenen ya başka bir kayıt ya da sabit bir değer olabilir.

Mantık talimatları sadece temel mantıksal işlemleri değil, aynı zamanda veri manipülasyonlarını da kolaylaştırır. Bu işlemler şunları içerir:

- **Kaydı Ters Çevirme:** Bir kaydın bitlerini tersine çevirerek veriyi değiştirebilirsiniz.
- **Kayıt Temizleme:** Kayıttaki tüm bitleri sıfırlayarak kaydı temizlersiniz.
- **Belirli Bitlerin Ayarlanması veya Temizlenmesi:** Kaydın yalnızca belirli bitlerini değiştirebilir ya da temizleyebilirsiniz.

Bu mantıksal işlemler, kayıtlarda daha karmaşık bit manipülasyonlarına imkan tanır ve sistemde veri üzerinde etkili bir kontrol sağlar.

### b) Aritmetik Talimatlar

PicoBlaze mikrodenetleyicisi, temel aritmetik işlemleri gerçekleştirebilmek için bayt çapında toplama ve çıkarma talimatları sunar. Bu talimatlar, tek baytlı işlemlerle sınırlı kalmaz; komutların birleşimiyle çok baytlı aritmetik işlemler de yapılabilir. Ayrıca, çarpma ve bölme gibi daha karmaşık işlemler de gerçekleştirilir.

- ADD ve ADDCY Talimatları

PicoBlaze mikrodenetleyicisi, iki temel ekleme talimatı sunar: ADD ve ADDCY. Bu talimatlar, sırasıyla taşıma (CARRY) olmadan ve taşıma ile iki 8 bitlik işlenenin toplamını hesaplamak için kullanılır. İlk işlenen her zaman bir kayıt konumudur, ikinci işlenen ise bir kayıt konumu ya da sabit bir değeri olabilir. Bu talimatların her ikisi de hesaplama sonucu olarak iki bayrak üzerinde değişiklik yapar: CARRY ve ZERO bayrakları.

Elde edilen toplam, 255'i aşarsa CARRY bayrağı set edilir. Örneğin, toplam 256 veya daha büyük bir değer olduğunda, bu durumda CARRY bayrağı aktif olur. Eğer toplam 0 veya 256 ise (veya taşıma ile sıfırlanan sX kaydı varsa), ZERO bayrağı set edilir.

ADDCY Talimatı, taşıma ile yapılan bir toplama işlemidir. Eğer önceki toplama işlemi sırasında CARRY bayrağı set edildiyse, ADDCY komutu bu taşıma değerini bir ekleme olarak dikkate alır. Bu komut, özellikle çok baytlı toplama işlemleri sırasında yaygın olarak kullanılır. Çünkü çok baytlı aritmetik işlemleri, her baytın toplama işlemi sırasında taşıma bayrağını dikkate alır ve doğru sonucun elde edilmesini sağlar.

- SUB ve SUBCY Çıkarma Talimatları

PicoBlaze mikrodenetleyicisi, çıkarma işlemleri için iki farklı talimat sunar: SUB ve SUBCY. Bu talimatlar, sırasıyla taşıma (CARRY) olmadan veya taşıma ile iki 8 bitlik işlenenin farkını hesaplamak için kullanılır. Çıkarma işlemi sırasında CARRY bayrağı, ödünç alma (borrow) olup olmadığını belirler ve işlemi etkiler.

İlk işlenen her zaman bir kayıt konumudur, ikinci işlenen ise bir başka kayıt konumu veya sabit bir değer olabilir. Çıkarma işlemi sonucunda, CARRY ve ZERO bayrakları değişir. Eğer çıkan fark, 0'dan küçükse, bu durumda CARRY bayrağı ayarlanır. Yani, çıkarma sırasında bir ödünç alma durumu oluşmuşsa, CARRY bayrağı aktif olur. Eğer fark 0 veya -256 olursa, ZERO bayrağı set edilir.

SUBCY talimatı, taşıma ile yapılan bir çıkarma işlemidir. Eğer önceki çıkarma işlemi sırasında CARRY bayrağı set edildiyse, SUBCY komutu bu taşıma değerini çıkarma işlemine dahil eder ve sonucu bir miktar daha küçük yapar. Bu talimat, özellikle çok baytlı çıkarma işlemleri için yaygın olarak kullanılır. Çünkü çok baytlı aritmetik işlemleri, her baytın çıkarılması sırasında taşıma bayrağını dikkate alır, bu da doğru sonucun elde edilmesini sağlar. Bu nedenle, SUBCY komutu genellikle çok baytlı çıkarma işlemleri ve büyük sayılarla yapılan hesaplamalar için önemlidir.

- Artırma/Azaltma

PicoBlaze mikrodenetleyicisi, doğrudan artırma veya azaltma işlemleri için özel bir talimat sunmaz. Bunun yerine, temel aritmetik talimatlar olan ADD ve SUB kullanılarak değerler artırılabilir veya azaltılabilir. Bu talimatlar, tek bir adımda iki 8 bitlik işlenen arasında toplam veya çıkarım gerçekleştirebilir. Eğer aynı anda çoklu kayıt değeri, yani 16 bitlik bir değeri artırmak veya azaltmak istiyorsanız, bu işlemi birden fazla talimat kullanarak gerçekleştirin. Her bir talimat, bir bayt değerini artırmak veya çıkarmak için tek bir işlem gerçekleştirir. Dolayısıyla, çok baytlı artırma veya azaltma işlemleri, sıralı olarak birden fazla talimatla tamamlanabilir ve her bir bayt için ayrı ayrı talimatlar uygulanabilir. Bu yöntem, büyük değerlerle yapılan aritmetik işlemlerde daha dikkatli ve ayrıntılı bir yaklaşımla işlemin tamamlanmasını sağlar.

- Olumsuzlama

PicoBlaze mikrodenetleyicisi, bir kayıt değerini olumsuzlamak için doğrudan bir talimat sunmaz. Ancak, bir kayıt değerinin tamamlayıcısını almak için, mevcut SUB (çıkarma) talimatı kullanılabilir. Bu yöntemle, belirli bir değerın olumsuz hali, o değerin negatifini almak suretiyle elde edilebilir. Yani, bir kayıt değerine karşılık gelen 2'nin tamamlayıcı alınarak değer olumsuzlaştırılır.

- Çarpma İşlemi

PicoBlaze mikrodenetleyici çekirdeği, özel bir donanım çarpanı içermemektedir. Ancak, çarpma işlemi gerçekleştirmek için mevcut aritmetik ve kaydırma talimatları kombin edilebilir. Bu talimatlar, ardışık ekleme veya kaydırma işlemleri kullanılarak çarpma işlemini simüle edebilir. Bu yöntem, donanım çarpanı olmadan çarpma işlemini yazılım düzeyinde gerçekleştirmeye olanak tanır.

- Bölme

Benzer şekilde, PicoBlaze mikrodenetleyicisinin özel bir donanım bölücüsü bulunmamaktadır. Ancak, bölme işlemi de aritmetik ve kaydırma talimatları ile yapılabilir. Bölme işlemi, kaydırma ve çıkarma işlemleri kombin edilerek simüle edilebilir. Bu işlem, bölme işlemine karşılık gelen sonuçları ve kalanı elde etmek için bir dizi ardışık işlem gerektirir.

- İşlem Yok (NOP)

PicoBlaze mikrodnetleyicisinin komut setinde, özel bir NOP (No Operation) talimatı bulunmamaktadır. Ancak, bir NOP talimatının tipik olarak yaptığı şey, hiçbir işlem yapmamaktır; yani, kayıtları veya bayrakları etkilemeden bir komut döngüsünü yürütür. Bunun yerine, bir yazmaç kendi değeriyle yüklenebilir. Bu işlem, yazmaç içeriğini veya bayrakları değiştirmez ve yalnızca bir işlem döngüsü gerektirir. NOP komutu, genellikle zamanlamayı kontrol etmek, kodun belirli kısımlarını senkronize etmek veya daha öngörülebilir yürütme zamanları elde etmek için kullanılır.

#### c) Test ve Karşılaştırma Talimatları

PicoBlaze mikrodnetleyicisi, önceki sürümlerinde bulunmayan iki yeni komut sunar: TEST ve COMPARE. Bu komutlar, mikrodnetleyicinin, bir kayıttaki bitleri test etme ve bir kayıt değerini başka bir kayıt veya sabit bir değerle karşılaştırma yeteneğini artırır. Her iki talimat da yalnızca ZERO ve CARRY bayraklarını etkiler, ancak kayıtlarda herhangi bir değişiklik yapmaz.

**Test Komutu:** İki işlenen arasında bit düzeyinde mantıksal AND işlemi gerçekleştirir. Bu komut, AND komutundan farklı olarak sadece ZERO ve CARRY bayraklarını etkiler ve herhangi bir kayıt üzerinde değişiklik yapmaz. İşlem, her bir bitin mantıksal olarak ya bir diğer kayıttaki veya sabitteki karşılık gelen bit ile AND yapılmasıyla gerçekleştirilir. Eğer tüm bit düzeyindeki AND işleminin sonucu sıfırsa, ZERO bayrağı ayarlanır. Eğer ikinci işlenenin herhangi bir biti '1' içeriyorsa, CARRY bayrağı, bir kayıttaki karşılık gelen bitin '1' olup olmadığını test eder. Geniş bir uygulamada, CARRY bayrağı, bir kaydın bitlerinin tek paritesini hesaplar. İkinci işlenen, bu testte bir maske görevi görür. Eğer ikinci işlenendeki bir bit '0' ise, bu bit test işlemine dahil edilmez; eğer '1' ise, o zaman ilgili bit sonuca dahil edilir.

**Karşılaştırma Komutu:** İki işlenen arasındaki 8 bitlik farkı hesaplar. Ancak, yalnızca ZERO ve CARRY bayraklarını etkiler ve kayıt üzerinde değişiklik yapmaz. Eğer her iki işlenen eşitse, ZERO bayrağı ayarlanır. Ayrıca, CARRY bayrağı, ikinci işlenenin birinci işlenenden daha büyük olduğunu belirtmek için kullanılır.

Bu komutlar, PicoBlaze mikrodnetleyicisine bit testi ve karşılaştırma yapma yeteneği ekleyerek, daha verimli ve esnek işlem yapma imkanı tanır.

PicoBlaze mikrodnetleyicisi, kaydırma ve döndürme işlemleri için çeşitli talimatlar sunar. Bu talimatlar, özellikle CARRY ve ZERO bayrakları üzerinde etkiler yaratır. Her kaydırma veya döndürme işlemi, bayrakları değiştirerek veri üzerinde işlem yapar. İşte bu talimatlar ve nasıl çalıştıkları:

#### d) Kaydırma ve Döndürme Talimatları

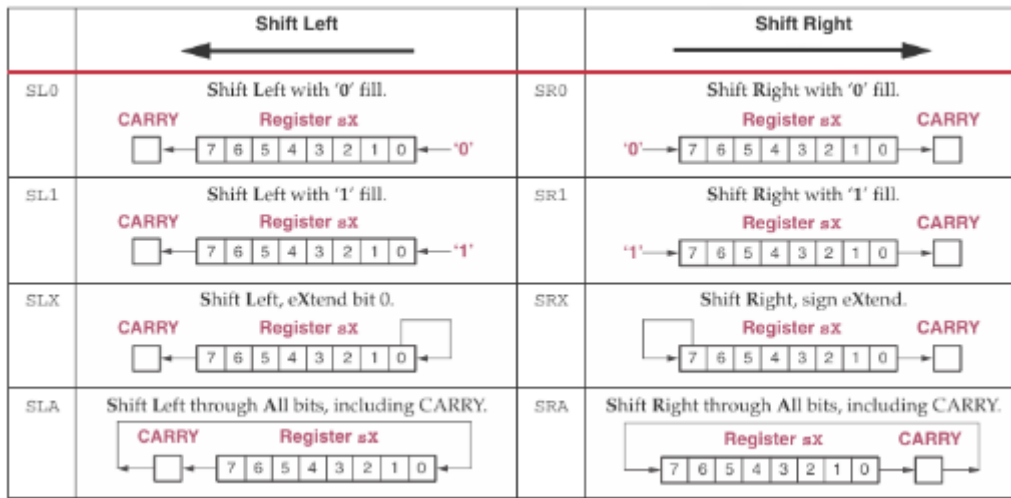
PicoBlaze mikrodnetleyicisi, kaydırma ve döndürme işlemleri için çeşitli talimatlar sunar. Bu talimatlar, özellikle CARRY ve ZERO bayrakları üzerinde etkiler yaratır. Her kaydırma veya döndürme işlemi, bayrakları değiştirerek veri üzerinde işlem yapar. İşte bu talimatlar ve nasıl çalıştıkları:

##### Kaydırma Talimatları

- **SL0 (sX):** Bu komut, sX kaydının içeriğini sola bir bit kaydırır. Kaydırma işlemi sırasında, en yüksek bit olan bit 7, CARRY bayrağına kaydırılır. En düşük bit olan bit ise 0 ile doldurulur. Bu işlem sırasında CARRY bayrağı güncellenir.
- **SR0 (sX):** SR0 komutu, sX kaydının içeriğini sağa bir bit kaydırır. Bu işlem sırasında, en düşük bit olan 0, CARRY bayrağına kayar. En yüksek bit ise 0 ile doldurulur. CARRY bayrağı, kaydırma işleminde etkilenir.
- **SL1 (sX) ve SR1 (sX):** SL1 ve SR1 komutları, SL0 ve SR0'a benzer şekilde çalışır. Ancak bu talimatlarda, kaydırılan boş bit konumu her zaman 1 ile doldurulur. Bu

sebeple, ZERO bayrağı her zaman 0 olur, çünkü kaydırma işlemiyle kaydedilen değere bir 1 eklenir ve bu da kaydın sıfırdan farklı olmasını sağlar.

- **SRX (sX)**: SRX komutu, aritmetik sağa kaydırma işlemi gerçekleştirir. Bu işlemde, işaret biti korunarak kaydırma yapılır. Kaydırma sırasında, en önemli bit olan bit 7 değişmez ve kaydırma işlemi sırasında tekrar bit 7'ye kopyalanır. Bu işlem, negatif sayılarla çalışırken işaret bitinin korunmasını sağlar.
- **SLX (sX)**: SLX komutu ise, kaydı sola kaydırır. Bu işlemde 0 biti çoğaltılır ve kaydın geri kalan kısmı da 0 ile doldurulur.
- **SLA (sX)**: SLA komutu, kaydı CARRY biti boyunca kaydırır. Bu işlem sırasında, CARRY bayrağındaki bit, kaydın en düşük biti olan bit 0'a geri beslenir. Böylece kaydırma işlemi gerçekleşir.
- **SRA (sX)**: SRA komutu, SLA komutuna benzer şekilde çalışır, ancak burada doğru bir kaydırma yapılır. CARRY bayrağı ve kaydırma işlemi arasında daha dikkatli bir yönetim sağlanır.



#### Döndürme Talimatları:

- **RL (sX)**: RL komutu, sX kaydındaki bitleri sola döndürür. Bu işlemde, kaydın en önemli biti (bit 7) en düşük bit olan bit 0'a kaydırılır ve CARRY bayrağına da kopyalanır.
- **RR (sX)**: RR komutu, sX kaydının içeriğini sağa döndürür. Bu işlem sırasında, kaydın en düşük biti (bit 0), CARRY bayrağına kaydırılır. En yüksek bit olan bit 7 de kaydırılma işlemiyle sağa doğru hareket eder.



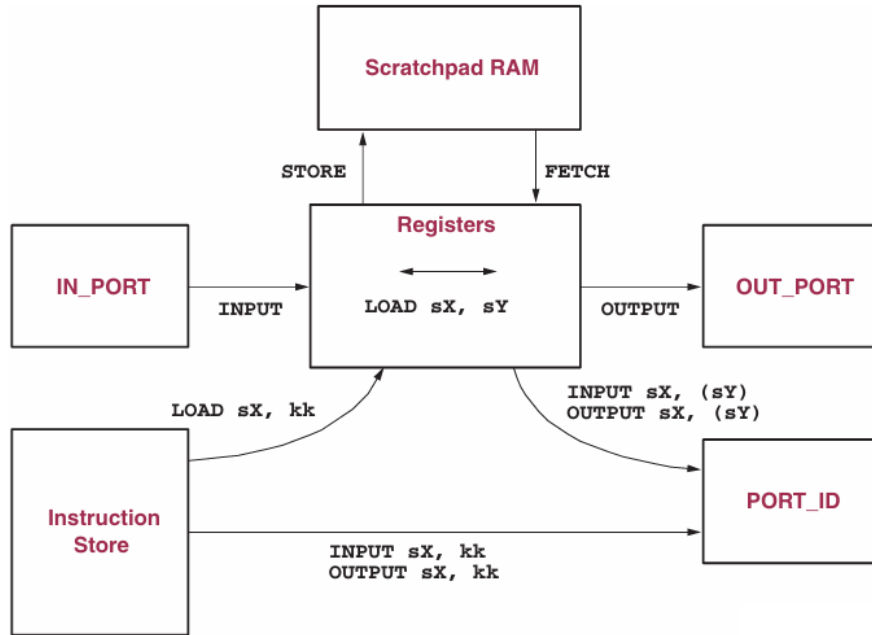
Bu talimatlar, kaydırma ve döndürme işlemleriyle verinin bit düzeyinde manipülasyonunu sağlar. CARRY bayrağının doğru bir şekilde yönetilmesi, bu işlemlerin başarılı bir şekilde yapılması için kritik öneme sahiptir.

## 2. Moving Data (Verileri Taşıma)

PicoBlaze mikrodnetleyicisinin veri taşıma talimatları, kayıtlara veri yüklemek veya kayıtlardan veri almak için kullanılır. Bu talimatlar, sistemdeki verilerin doğru şekilde hareket etmesini sağlar. İşte bu talimatların açıklamaları:

- **LOAD sX, sY Komutu:** Bu komut, bir kayıttan diğerine veri taşır. Kaydın sY içeriği, sX kaydına yüklenir. Yani, sX kaydı, sY kaydındaki verileri alır. Bu işlem, sadece kayıtlar arasında veri taşıma işlemi yapar ve sistem bayrakları üzerinde herhangi bir değişiklik oluşturmaz.
- **LOAD sX, kk Komutu:** Bu komut, bir sabit değeri kayda yükler. kk bir sabit sayıyı temsil eder ve bu sabit, belirtilen sX kaydına yüklenir. Bu komut da CARRY veya ZERO bayraklarını etkilemez, sadece belirtilen kayda sabit bir değer yükler.
- **STORE ve FETCH Talimatları:** Bu talimatlar, verileri bir kayıt dosyasından (register file) bir başka hafıza alanına, örneğin Scratchpad RAM'e taşır. STORE talimatı verileri kayıttan RAM'e yazarken, FETCH talimatı RAM'den kayda veri okur. Bu işlemler de bayrakları etkilemez.
- **INPUT ve OUTPUT Talimatları:** PicoBlaze mikrodnetleyicisinin giriş/çıkış işlemleri de veri taşıma sürecine dahildir. Bir INPUT işlemi sırasında, IN\_PORT giriş portundan gelen veriler kayıtlardan birine yazılır. Benzer şekilde, bir OUTPUT komutunda, OUT\_PORT çıkış portuna yazılan veriler, kayıtlardan birinden kaynaklanır. PORT\_ID ise giriş veya çıkış portu adresini belirtir ve bu adres, kayıtlardan birinden ya da sabit bir değerden alınır.

Bu talimatlar herhangi bir bayrağı etkilemediği için, yalnızca veri taşıma işlemi gerçekleşir.



## 3. Program Akış Kontrolü

Program Sayacı (PC), PicoBlaze mikrodnetleyicisinin yürütülecek olan bir sonraki talimatı belirleyen önemli bir bileşendir. PC, varsayılan olarak talimat deposundaki bir sonraki komut adresini işaret eder ve bu şekilde programın sırasıyla akışını sağlar (yani,  $PC = PC + 1$ ). Ancak, Program Sayacına doğrudan erişim mümkün değildir. Bunun yerine, program akışını değiştirebilen birkaç talimat vardır.

PicoBlaze mikrodnetleyicisinde, üç ana talimat türü -JUMP, CALL ve RETURN- Program Sayacını farklı bir değere yükleyerek varsayılan akışta değişiklik yapabilir. Bu talimatlar, belirli bir koşul sağlandığında (örneğin, bir bayrağın durumu) yürütülür. Bu, programın akışını belirli durumlara göre yönlendirmek için kullanılır.

Ayrıca, kesme olayları da program akışını değiştirebilir. Kesme, belirli bir koşul gerçekleştiğinde mikrodenetleyicinin kontrolünü değiştirerek, belirli bir talimatı yürütmeye başlatabilir. Bir diğer önemli etki de sıfırlama olaylarıdır. Bir sıfırlama durumu meydana geldiğinde, program sıfırlanır ve yürütme baştan başlar.

JUMP, CALL ve RETURN komutları, koşullarına göre çalışır. Bu koşullar, genellikle bayrakların (CARRY veya ZERO bayrakları gibi) durumu ile ilişkilidir. Bu bayrakların ayarlanıp ayarlanmadığına göre komutlar koşullu şekilde yürütülür. Yani, bir komut yalnızca belirli koşullar altında çalıştırılır. Eğer koşul belirtilmemişse, talimat koşulsuz olarak çalıştırılır ve program akışı değiştirilmeden bir sonraki komuta geçilir.

| Koşul  | Açıklama                                |
|--------|-----------------------------------------|
| <none> | Talimatı koşulsuz olarak yürütün.       |
| C      | CARRY bayrağı set ise talimatı yürüt.   |
| NC     | CARRY bayrağı reset ise talimatı yürüt. |
| Z      | ZERO bayrağı set ise talimatı yürüt.    |
| NZ     | ZERO bayrağı reset ise talimatı yürüt.  |

### JUMP, CALL ve RETURN Komutları

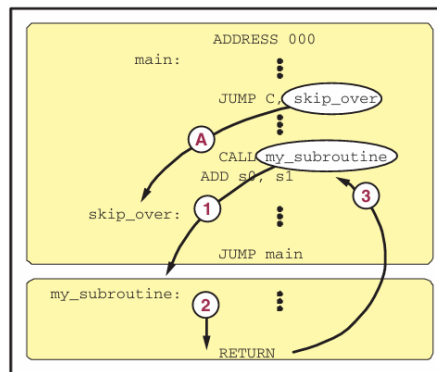
PicoBlaze mikrodenetleyicisi, program akışını kontrol etmek için çeşitli komutlar sunar. Bu komutlar arasında JUMP, CALL, ve RETURN komutları, programın akışını yönlendirmek için kullanılır ve her biri belirli koşullara bağlı olarak çalışır. Bu komutlar, programın akışını değiştirirken, özellikle bayraklar ve yığın ile etkileşimi önemlidir.

#### JUMP Komutu:

JUMP komutu, programın akışını belirli bir etikete atlar. Bu komut koşulludur ve yalnızca belirtilen koşul sağlandığında yürütülür. Koşul sağlanmadığında, komut yalnızca iki saat döngüsü kadar gecikmeye neden olur, ancak programın diğer kısmına etkisi olmaz; yani bayraklar veya kayıtlar değişmez.

Eğer koşul sağlanırsa, JUMP komutu, PC'yi programcı tarafından hesaplanan etikete yükler. Bu, PicoBlaze mikrodenetleyicisinin program akışını belirli bir adrese yönlendirmesini sağlar. Atlamalar mutlaktır, yani göreceli sıçramalar veya hesaplanan atlamalar desteklenmez.

Örneğin, bir JUMP C, skip\_over komutu, önce CARRY bayrağını kontrol eder. Eğer CARRY bayrağı ayarlanmışsa, skip\_over etiketinin adresi PC'ye yüklenir ve program akışı bu adrese yönlendirilir. Bu komut, ZERO ve CARRY bayraklarını etkilemez.



CALL ve RETURN komutları, programın akışını geçici olarak bir alt programa yönlendiren ve ardından geri dönen komutlardır. Bu komutlar, JUMP komutundan farklı olarak, bir alt yordam çağırma ve sonrasında geri dönmeyi sağlar.

### CALL Komutu:

CALL komutu, belirli bir koşul sağlandığında çalışır ve koşul sağlanmadığında sadece iki saatlik bir döngüye neden olur. Koşul sağlandığında, mevcut PC değeri, CALL/RETURN yığınının üstüne itilerek kaydedilir. Daha sonra, montajcı tarafından hesaplanan etikete PC yüklenir ve program akışı bu etikete yönlendirilir.

PicoBlaze mikrodenetleyicisi, CALL komutunu işledikten sonra, alt yordamı çağırmaya devam eder. Bu, RETURN komutu karşılaşılan kadar devam eder. CALL komutunun ardından mutlaka bir RETURN komutu bulunmalıdır.

### RETURN Komutu:

RETURN komutu, bir alt yordamın sonlandırılmasını sağlar. RETURN komutu da koşulludur ve yalnızca belirtilen koşul sağlandığında çalışır. Koşul sağlandığında, CALL/RETURN yığınının üst kısmı açılır ve orijinal CALL komutunun hemen ardından program akışı geri döner.

Bu komutun işleyişi sırasında, SIFIR ve CARRY bayrakları, çağrılan alt yordamda yapılan işlemlerle değiştirilebilir, ancak doğrudan CALL veya RETURN komutları bayrakları etkilemez. Eğer CALL komutu çalıştırılmazsa, bayraklar değişmez.

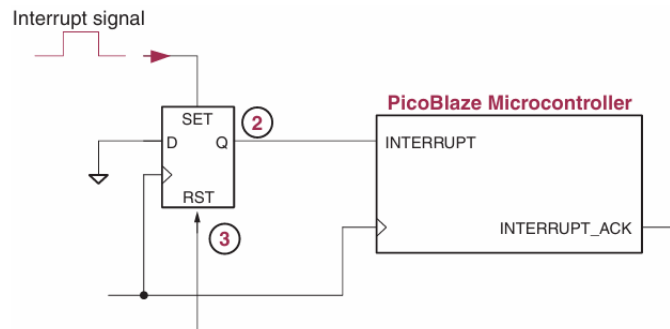
### Kesintiler (Interrupts):

PicoBlaze işlemcisi, program akışını keserek belirli olaylara yanıt vermek için bir kesme (interrupt) mekanizması sağlar. İşlemci, yalnızca tek bir kesme giriş sinyali (INTERRUPT) ile çalışır. Ancak, uygulamanızda birden fazla kesme sinyali gerekiyorsa, bu sinyalleri birleştirerek tek bir INTERRUPT sinyali oluşturmak için basit FPGA mantığı kullanılabilir.

#### Kesme Mekanizmasının Etkinleştirilmesi ve Devre Dışı Bırakılması

Sıfırlamadan sonra, INTERRUPT girişi varsayılan olarak devre dışıdır. Kesme mekanizmasını etkinleştirmek için ENABLE INTERRUPT komutu kullanılmalıdır. İşlemin herhangi bir noktasında kesme mekanizmasını devre dışı bırakmak için ise DISABLE INTERRUPT komutu verilir.

Etkinleştirildikten sonra, bir INTERRUPT olayının tanınabilmesi için kesme sinyali en az iki saat döngüsü boyunca uygulanmalıdır. Bu sinyal, işlemcinin bir INTERRUPT Olayı oluşturmasını sağlar. Kesme mekanizmasının etkin olduğu bir durumda, işlemci o anda yürütülmekte olan talimatı tamamlar ve ardından CALL 3FF komutunu otomatik olarak yürütür.



#### CALL 3FF ve Kesme İşlem Süreci

CALL 3FF komutu, işlemcinin program belleğinin son adresine (3FF) bir alt yordam çağırması yapmasını sağlar. Bu adresteki yönerge, kesme işlemi sırasında ne yapılacağını belirler. Tipik bir kullanımda, bu adres bir kesme hizmeti rutini (Interrupt Service Routine - ISR) konumuna bir atlama talimatını barındırır.

Kesinti sırasında PicoBlaze mikrodenetleyicisi, aşağıdaki işlemleri otomatik olarak gerçekleştirir:

1. ZERO ve CARRY bayraklarının mevcut durumunu korur.
2. Daha fazla kesintiyi devre dışı bırakır.
3. Mevcut Program Sayacı (PC) değerini CALL/RETURN yığınınına kaydeder.

Kesme işlemi sırasında INTERRUPT\_ACK sinyali üretilir. Bu sinyal, kesmenin tanındığını göstermek için kesinti olayının ikinci döngüsünde gönderilir. Ayrıca, harici kesintileri temizlemek için kullanılabilir.

#### Kesinti Hizmetinin Sonlandırılması: RETURNI Komutu

Kesme işlemi tamamlandığında, program akışını kesintinin gerçekleştiği noktadan devam ettirmek için özel bir RETURNI komutu kullanılır. RETURNI komutu şu işlevleri yerine getirir:

1. CALL/RETURN yığınınındaki orijinal PC değerini geri yükler ve program akışını kesintiden önceki noktaya döndürür.
2. ZERO ve CARRY bayraklarının durumunu geri yükler.
3. Gelecekteki kesintilerin etkin olup olmayacağını belirler.

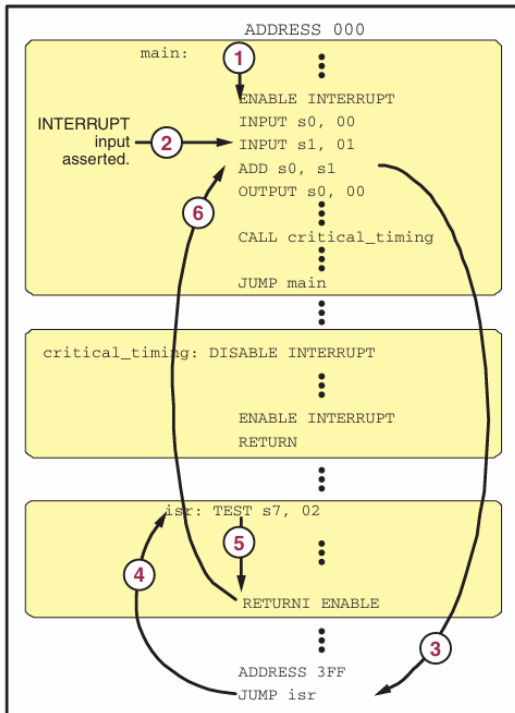
Kesintiler, kesme hizmeti rutini boyunca devre dışı kalır ve ancak RETURNI komutunun ardından yeniden etkinleştirilebilir.

#### Kesme Gerekmediğinde

Uygulamanız bir kesme mekanizmasına ihtiyaç duymuyorsa, INTERRUPT sinyali düşük durumda (LOW) bağlanabilir. Bu durumda, kesme işlevi devre dışı kalır ve program belleğindeki tüm 1.024 bellek konumu uygulama kodu için kullanılabilir. PicoBlaze işlemcisindeki kesme mekanizması, olaylara hızlı yanıt verebilmek için tasarlanmıştır.

#### Kesme Girişinin Yönetimi ve Kesinti Süreci

PicoBlaze mikrodenetleyicisi, kesme (interrupt) mekanizması ile program akışını keserek belirli olaylara yanıt verebilir. Ancak, kesme girişinin tanınabilmesi için bazı özel şartlar ve süreçler gereklidir. Bu bağlamda, kesintilerin zamanlama açısından kritik işlevlerde nasıl yönetileceğini ve bir kesinti olayının nasıl işlendiğini açıklayan detaylı bir örnek üzerinden ilerleyeceğiz.



#### Kesme Girişinin Tanınması

1. **Kesmenin Etkinleştirilmesi:** Varsayılan olarak, INTERRUPT girişi devre dışıdır. Kesme girişinin tanınması için ENABLE INTERRUPT komutunun yürütülmesi gerekir. Bu komut yürütüldüğünde, INTERRUPT\_ENABLE bayrağı ayarlanır ve kesintiler etkinleştirilir.
2. **Kesmenin Devre Dışı Bırakılması:** Kesintiler, zamanlama açısından kritik işlevlerde mutlak öngörülebilirlik sağlamak için geçici olarak devre dışı bırakılabilir. Kritik işlev tamamlandığında kesme girişini yeniden etkinleştirmek mümkündür.

## Kesinti Sürecinin Adımları

### Adım 1: Kesme Girişi ve Program Akışı

Bu örnekte, PicoBlaze mikrodenetleyicisi kesme mekanizmasının etkin olduğu bir durumda, INPUT s1, 01 komutunu yürütmektedir. INTERRUPT girişine bir kesme sinyali geldiğinde işlemci, mevcut komutun yürütülmesini tamamlar ve ardından kesme girişini değerlendirir.

### Adım 2: Kesmenin Tanınması ve Hazırlık

Kesme girişinin tanınmasıyla birlikte:

- Kesme sinyali değerlendirilir ve işlemcinin yürütmekte olduğu komutun ardından gelen talimat (örneğin, ADD s0, s1) önlenir.
- Mevcut Program Sayacı (PC) değeri, CALL/RETURN yığınınına kaydedilir.
- ZERO ve CARRY bayraklarının mevcut durumları korunur.
- INTERRUPT\_ENABLE bayrağı temizlenir ve daha fazla kesinti devre dışı bırakılır.
- PC'ye kesme vektörünün bulunduğu bellek konumu (3FF) yüklenir.

### Adım 3: Kesme Hizmeti Rutininin Çağrılması

Kesme vektörü, program belleğinin en son konumunda yer alır. Bu konum genellikle bir JUMP isr komutunu barındırır ve kesme hizmeti rutini (ISR) burada başlar. ISR, kesmenin meydana geldiği anda programın nasıl tepki vereceğini tanımlar.

### Adım 4: Kesme Hizmeti Rutininin Tamamlanması

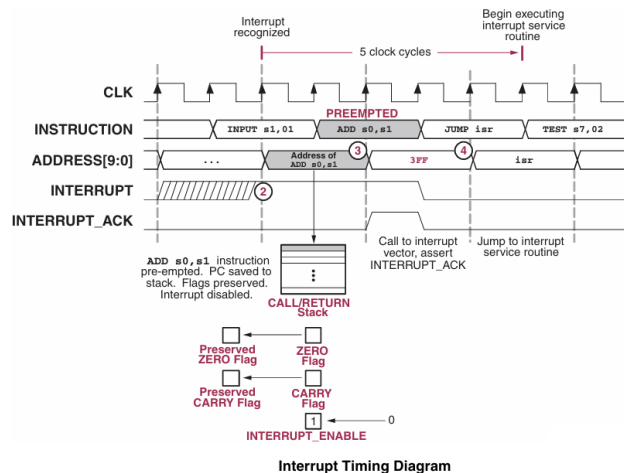
Kesinti işleme tamamlandığında, kesme hizmeti rutini aşağıdaki şekilde sonlandırılır:

- RETURNI komutu kullanılır. Bu komut, INTERRUPT\_ENABLE bayrağını tekrar etkinleştirir ve ZERO ile CARRY bayraklarının korunmuş durumlarını geri yükler.
- RETURNI ayrıca, CALL/RETURN yığınınındaki orijinal PC değerini geri yükler. Böylece işlemci, kesinti tarafından engellenen talimatı yürütmeye devam eder.

### RETURNI Komutunun Önemi

- RETURNI ENABLE:** Kesinti hizmetinden döner ve INTERRUPT girişini yeniden etkinleştirir.
- RETURNI DISABLE:** Kesinti hizmetinden döner ancak INTERRUPT girişini devre dışı bırakır.

RETURNI, ZERO ve CARRY bayraklarını geri yükleyerek kesme öncesi durumu tamamen geri kazandırır. Ayrıca, program akışı, kesmenin meydana geldiği talimatın hemen ardından devam eder.



Kesme işlemi, PicoBlaze mikrodeneleyicisinin program akışını değiştiren önemli bir özelliktir. Kesme etkinleştirildiğinde, INTERRUPT girişi, ADDRESS veri yolunun değerini değiştiren aynı saat döngüsünde tanınır. Bu tanıma, kesme sinyalinin girişinin ADD s0, s1 komutunun adresiyle aynı anda işlendiği anlamına gelir. Kesme sinyali alındığında, program akışı hemen kesme vektörüne yönlendirilir.

Kesme tanındığında, ADD s0, s1 komutunun adresi ADDRESS veri yolunda görünür ve bu adres, CALL/RETURN yığınınına itilir. Bu işlemle birlikte, mevcut komutun yürütülmesi durdurulur ve program sayacı (PC) kesme vektörüne geçiş yapar.

Aynı anda, INTERRUPT girişi devre dışı bırakılır ve kesme işlemi engellenir. ZERO ve CARRY bayrakları da korunur, yani kesme sırasında bayraklar değişmeden kalır ve kesme hizmeti rutini (ISR) tamamlandıktan sonra eski durumlarına geri döner.

Kesme sırasında ADD s0, s1 komutu hemen yürütülmez. Bunun yerine, PicoBlaze mikrodeneleyicisi 0x3FF adresindeki kesme vektörüne bir çağrı yapar. Bu kesme vektörü, programın kontrol akışını yeniden yönlendirir ve mikrodeneleyicinin kesme hizmeti rutini (ISR) işlemesine olanak tanır.

Zamanlama açısından kritik işlevlerde veya kesin zamanlama gereksinimlerinin bulunduğu durumlarda, kesme işlemi istenmeyen bir durum olabilir. Kritik bir prosedür sırasında kesmenin devreye girmesi, programın doğru ve öngörülebilir şekilde çalışmasını engelleyebilir. Bu tür durumlar için, kesmelerin geçici olarak devre dışı bırakılması gereklidir.

Kritik işlevler sırasında, kesmeleri geçici olarak devre dışı bırakmak için DISABLE INTERRUPT komutu kullanılabilir. Bu komut, kesme işlemlerinin kritik bölümde yer almasını engeller, böylece kesintilerden bağımsız bir işlem yapılmasını sağlar. Kritik prosedür tamamlandığında, kesmelerin tekrar etkinleştirilmesi için ENABLE INTERRUPT komutu kullanılır. Bu komut, daha önce devre dışı bırakılmış olan kesme girişini yeniden aktif hale getirir ve PicoBlaze mikrodeneleyicisi kesmeleri yeniden tanıyabilir.

## **Scratchpad RAM (Kazı Tahtası RAM)**

PicoBlaze mikrodeneleyicisi, 64 bayt kapasitesinde bir karalama defteri RAM içerir ve bu RAM, STORE ve FETCH olmak üzere iki temel talimat aracılığıyla, verileri bir veri kaydı ile RAM arasında taşıyabilir. STORE talimatı, veriyi bir kayıt içerisinden RAM'e yazar, FETCH ise RAM'deki veriyi bir kayda okur. PicoBlaze mikrodeneleyicisi, her iki talimatla da hem doğrudan hem de dolaylı adresleme yöntemlerini destekler.

Karalama defteri RAM, yalnızca Spartan-3, Spartan-6® ve Virtex-6® FPGA modelleriyle uyumludur ve RESET olayı sırasında bu bellek yapısına herhangi bir etkisi olmaz. Bu sayede, işlem sırasındaki veriler sabit kalır.

### **Adresleme Modları**

PicoBlaze mikrodeneleyicisi, veriye erişim sağlamak için iki farklı adresleme yöntemi sunar: doğrudan adresleme ve dolaylı adresleme.

#### **Doğrudan Adresleme**

RAM'deki belirli bir konuma doğrudan sabit bir değer atamayı sağlar. Bu durumda, RAM adresi doğrudan belirtilir. Örneğin, aşağıdaki kodda, STORE komutu kullanılarak sX kaydındaki veri RAM konumu 04'e yazılır ve daha sonra FETCH komutu ile bu değer sX kaydına okunur.

```
scratchpad_transfers:
STORE sX, 04
FETCH sX, 04
```

## Dolaylı Adresleme

RAM adresinin sabit bir değerden değil, belirli bir kayıttaki değerden alındığı bir tekniktir. Bu durum, doğrudan adreslemeden daha esnek bir yaklaşım sunar. Çünkü adres, bir kayıttaki veriye göre hesaplanabilir ya da değiştirilebilir. Yani, RAM adresi dinamik olarak program sırasında belirlenebilir.

Örneğin, aşağıdaki kod dolaylı adresleme kullanımını gösteren bir kod örneği sunar. ram\_address kaydına 64'ün başlangıç değeri olan ram\_locations yüklenir ve daha sonra bu adres üzerinden ram\_data kaydına initial\_value olan 0 değeri yazılır. Adres, her döngüde 1 azaltılarak, tüm RAM konumları sıfırlanana kadar işlem devam eder.

```
NAMEREG s0, ram_data
NAMEREG s1, ram_address
CONSTANT ram_locations, 40
CONSTANT initial_value, 00
LOAD ram_data, initial_value
LOAD ram_address, ram_locations
ram_fill: SUB ram_address, 01
STORE ram_data, (ram_address)
JUMP NZ, ram_fill
```

## Yığın İşlemleri (Stack Operations)

PicoBlaze mikrodenetleyicisinin bir "çağrı ve dönüş" yığınının sahip olmasına rağmen, doğrudan bir veri yığını yoktur. Bu, PicoBlaze'in sınırlı donanım kaynaklarıyla işlevselliği nasıl genişlettiği ve bazı karmaşık işlemleri nasıl yönettiğiyle ilgilidir. Diğer bazı mikrodenetleyici ve işlemci mimarilerinde, alt rutinler çağrıldığında veya kesintiler gerçekleştiğinde, belirli kayıtlar (register) değerleri bir yığına itilir veya bir veri yığına kaydedilir. Bu işlem, fonksiyonlar arasında veri geçişini veya işlem sırasını yönetmek için oldukça faydalıdır.

PicoBlaze'de benzer bir işlem, karalama defteri RAM kullanılarak gerçekleştirilir. Yani, PicoBlaze mikrodenetleyicisi, fiziksel olarak bir "yığın" alanına sahip olmasa da, karalama defteri RAM'inde bazı özel bellek konumlarını ayırarak bu işlevi yerine getirir. Bu tür bir bellek yönetimi, PicoBlaze'in oldukça verimli bir şekilde alt rutin çağrıları ve kesintileri yönetmesini sağlar.

## Yığın Koruma ve Yönetimi

PicoBlaze'de bir yığın işlemi, belirli kayıtların korunması için yapılır. Bu kayıtlar genellikle işlem sırasında geçici veriler taşıyan veya alt rutinler tarafından kullanılan yazmaçlar (registers) olabilir. Karalama defteri RAM, bu tür verilerin korunması için ideal bir alan sağlar. Örneğin, bir alt rutine girmeden önce bir kaydın değeri, karalama defteri RAM'ine yazılabilir ve alt rutin tamamlandıktan sonra bu değer geri yüklenebilir.

```
NAMEREG sF, stack_ptr
LOAD sF, 20
my_subroutine:
    CALL push_s0
    CALL pop_s0 RETURN
RETURN

push_s0:
    STORE s0, stack_ptr
    ADD stack_ptr, 01
    RETURN

pop_s0:
```

Bu kodda, push\_s0 ve pop\_s0 işlemleri, s0 kaydının geçici olarak saklanması ve geri yüklenmesi için birer işlev gibi çalışır. Her iki işlem de, bir stack pointer kullanarak karalama defteri RAM'inde saklama ve geri alma işlemlerini gerçekleştirir.

## FIFO İşlemleri

FIFO (First-In-First-Out), bir veri yapısıdır ve verilerin sırasıyla işlendiği, ilk giren verinin ilk çıkan veri olduğu prensibe dayanır. PicoBlaze mikrodenetleyicisinde FIFO işlemleri, scratchpad RAM üzerinde iki ayrı işaretçi kullanılarak yapılabilir. Bu işaretçilerden biri, RAM'e yazılan verileri izlerken, diğeri RAM'den okunan verileri izler. Bu şekilde FIFO işlevselliği, iki işaretçi aracılığıyla uygulanır.

FIFO işlemleri, genellikle veri akışının sıralı bir şekilde yönetilmesi gerektiği durumlarda kullanılır. Örneğin, bir sensörden gelen veri okuma işlemi ile bir işlemciye veri yazma işlemi arasında sıralı bir düzenin sağlanması gerektiğinde FIFO yapısı tercih edilir.

PicoBlaze mikrodenetleyicisinde FIFO işlemleri şu şekilde gerçekleştirilebilir:

1. **İki İşaretçi Kullanımı:** Bir işaretçi, FIFO kuyruğuna veri yazarken, diğeri işaretçi ise kuyruğundan veri okur. Bu, veri akışının kontrolünü sağlar.
2. **Veri Yazma ve Okuma:** FIFO'ya veri yazma işlemi, bir işaretçi tarafından takip edilir ve bu işaretçi, veriyi yazdıktan sonra konumunu ilerletir. Veri okuma işlemi, ikinci işaretçi tarafından yapılır ve okunan veri, işaretçi konumunu geri alır.

Bu FIFO işlemleri, PicoBlaze mikrodenetleyicisinin scratchpad RAM'i üzerinde oldukça etkili bir şekilde gerçekleştirilebilir. FIFO yapısı, veri işlemlerinin sıralı bir şekilde yönetilmesi gerektiğinde kullanılır ve bu sayede verilerin doğru sırayla işlenmesini sağlar.

## Input ve Output (G/Ç) Portları

PicoBlaze mikrodenetleyici, 256 giriş portu ve 256 çıkış portunu destekler. Bu portlar, giriş/çıkış (I/O) portları oluşturmak üzere birleştirilebilir. Giriş ve çıkış işlemlerinde kullanılan arayüz sinyalleri şu şekilde açıklanabilir:

- **PORT\_ID[7:0]:** Bu çıkış portu, hem GİRİŞ hem de ÇIKIŞ işlemleri sırasında port kimlik numarasını veya port adresini belirtir.
- **IN\_PORT[7:0]:** INPUT işlemleri sırasında giriş verilerini alır ve yakalar.
- **OUT\_PORT[7:0]:** OUTPUT işlemleri sırasında çıkış verilerini taşır ve sunar.
- **READ\_STROBE:** İki döngülü INPUT işleminin ikinci döngüsünde yüksek (High) seviyeye çıkarak veri okuma işlemini onaylar.
- **WRITE\_STROBE:** İki döngülü OUTPUT işleminin ikinci döngüsünde yüksek (High) seviyeye çıkarak veri yazma işlemini onaylar.

## Zamanlama Kritik Tasarımlar İçin İpuçları

Zamanlama açısından hassas tasarımlarda, PORT\_ID ve veri yolları için zamanlama kısıtlamaları, iki saat döngüsüne izin verecek şekilde ayarlanmalıdır. Ancak, yalnızca okuma ve yazma flaşları tek bir saat döngüsüyle sınırlanmalıdır. En iyi performans elde etmek ve zamanlama kısıtlamalarını basitleştirmek için, mümkün olduğunda bir işlem hattı (pipeline) kaydı eklemek önerilir. Bu tür bir yaklaşım, arayüz mantığını hem kompakt hem de yüksek performanslı tutar.

## PORT\_ID Portu

8 bitlik PORT\_ID portu, INPUT ve OUTPUT işlemleri için port adresini veya port kimlik numarasını sağlar. PORT\_ID portu, iki saat döngüsü boyunca geçerlidir. Bu, arayüz kod çözme mantığı veya asenkron RAM bağlantıları gibi işlemler için yeterli süre tanır. Aynı şekilde, iki döngülü işlem, senkron RAM'den (örneğin, blok RAM) okuma işlemlerine olanak tanır.

## Adresleme Modları

GİRİŞ ve ÇIKIŞ işlemleri, hem doğrudan hem de dolaylı adreslemeyi destekler:

- **Doğrudan Adresleme:** Port adresi, 8 bitlik sabit bir değer olarak sağlanır.
- **Dolaylı Adresleme:** Port adresi, 16 veri kaydından birinin içeriği olarak tanımlanır. Bu yöntem, özellikle ardışık port adreslerine sahip bir blok bellek veya çevre birimini erişmek için idealdir.

## Harici Çevre Birimlerinin Eklenmesi

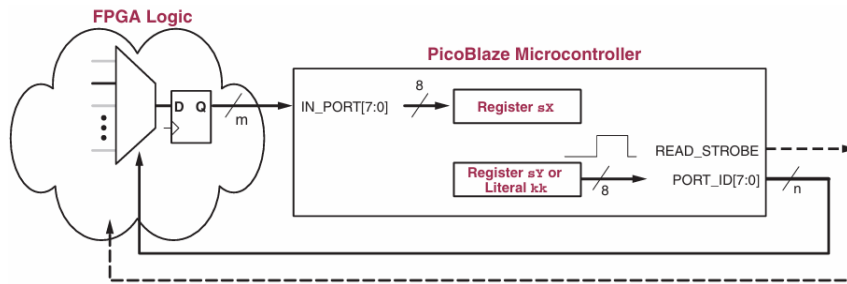
PicoBlaze mikrodenetleyiciye harici çevre birimleri eklemek oldukça basittir. Bununla birlikte, uygulama için gerekli minimum mantık ile PORT\_ID değerinin kod çözülmesi bir zorluk oluşturabilir. Bu zorluk, giriş, çıkış veya çift yönlü port sayısına bağlıdır. Kod çözme mantığı, ilgili portların sayısına göre optimize edilmelidir.

## Doğru Mantık Tasarımı ve Sentez

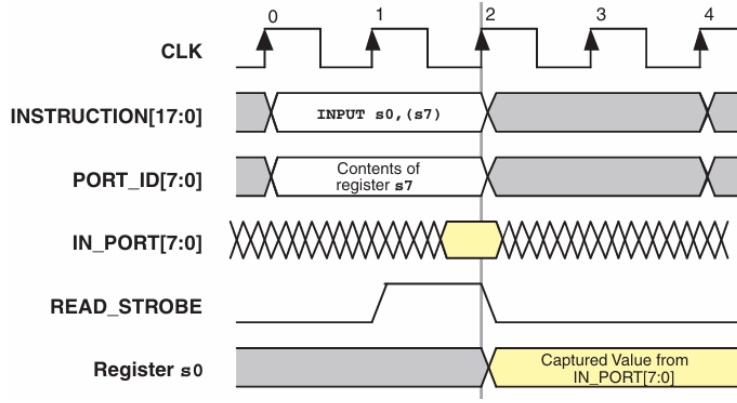
Bir mantık sentez aracı kullanıldığında, kaynak kodun gereğinden daha karmaşık bir devre tanımlamadığından emin olunmalıdır. Ayrıca, sentez aracının, tasarımcı tarafından amaçlanan mantığı uyguladığı kontrol edilmelidir. Böylece, performansı ve tasarım verimliliğini optimize eden kompakt ve doğru bir arayüz tasarımı elde edilebilir.

## INPUT Operations (GİRİŞ İşlemleri)

Bir INPUT işlemi sırasında, PicoBlaze™ mikrodenetleyici, IN\_PORT giriş bağlantı noktasında sunulan verileri 16 veri kaydından birine aktarır. Bu işlemde, hedef veri kaydı sX kaydı olarak tanımlanır. Giriş kaynağını belirlemek için PORT\_ID portu kullanılır; bu port, bir sY kaydı veya 8 bitlik bir sabit ile tanımlanabilir. PORT\_ID, genellikle bağlantı noktasındaki belirli bitlerin bir kısmını çoklayıcılar aracılığıyla kullanarak giriş kaynağını seçer. Çoklayıcının boyutu, sistem performansı üzerinde doğrudan etkili olup, kullanılabilecek giriş kaynaklarının sayısı ile ilişkilidir.



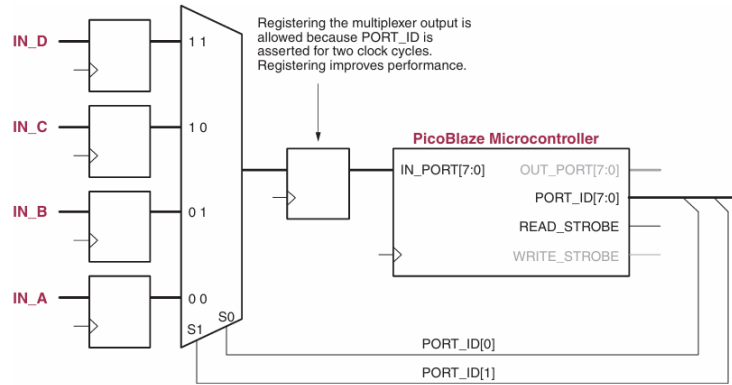
INPUT işlemi iki saat döngüsünden oluşur. İşlemin ikinci döngüsünde READ\_STROBE sinyali yükseltilir. Bu sinyal, genellikle uygulamalarda nadiren kullanılsa da, PicoBlaze'in verileri başarıyla elde ettiğini belirtmek için kullanılır. FIFO (İlk Giren İlk Çıkar) gibi veri yapılarında, bu sinyalin kritik bir önemi vardır.



INPUT işlemi sırasında, s7 kaydındaki PORT\_ID değeri, giriş kaynağını belirler ve ilgili harici FPGA mantığı tarafından çözülür. Bu işlem sonucunda, hedef veri s0 kaydına aktarılır. İlgili veri, komutun ikinci saat döngüsünün sonunda IN\_PORT bağlantı noktasından alınır. Verinin bu döngü sırasında mevcut ve stabilize olması, doğru veri yakalama için önemlidir.

### PORT\_ID ve Performans Optimizasyonu

PORT\_ID portu, iki saat döngüsü boyunca geçerlidir. Bu süre, giriş veri çoklayıcısının gerektiğinde kayıtlanmasını mümkün kılar. Giriş yollarının kaydedilmesi sistem tasarımını basitleştirir ve zamanlama sorunlarını önler. Kaydedilmeyen giriş yolları, genellikle sistem saat hızlarının düşmesine neden olur. Bu nedenle, performansı korumak için PORT\_ID'dan IN\_PORT'ye kadar olan yolda bir kaydın mevcut olması önerilir.



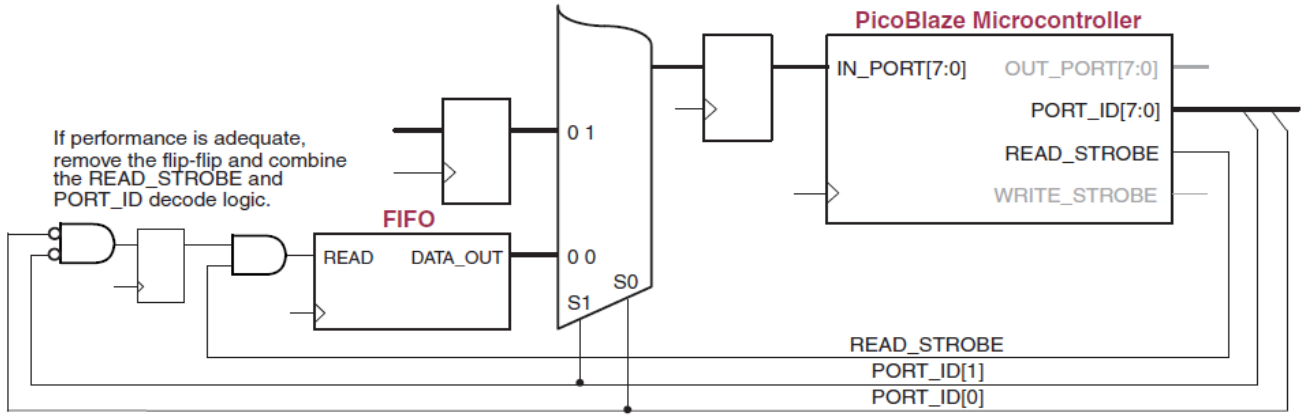
### Az Sayıda Giriş Kaynağı İçeren Uygulamalar

Eğer sistemde 32 veya daha az giriş kaynağı bulunuyorsa, yukarıdaki şekil gibi tek bir çoklayıcı kullanılması idealdir. Bu yöntem, tüm giriş sinyallerini IN\_PORT giriş bağlantı noktasına bağlamak için etkili bir çözüm sunar. Çoklayıcının performansını optimize etmek için özel MUXF5, MUXF6, MUXF7 ve MUXF8 yapılarının kullanımı önerilir. Bu optimizasyon, daha yüksek verimlilik ve doğru sentez sonuçları elde edilmesine yardımcı olur.

### READ\_STROBE ve FIFO Entegrasyonu

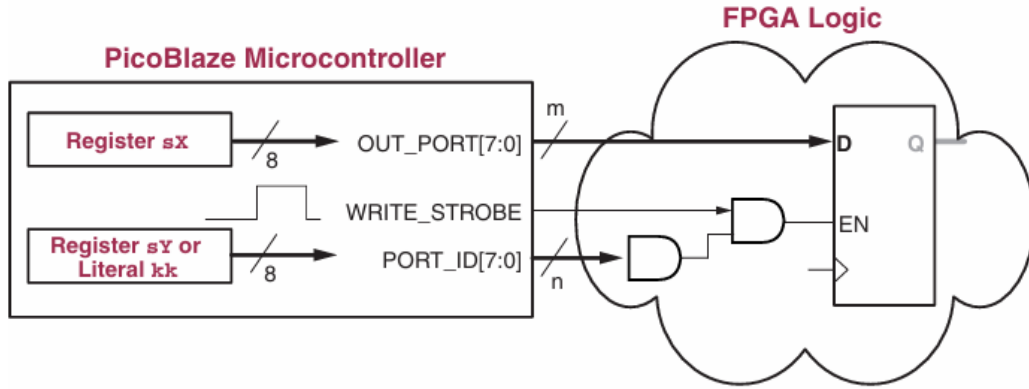
Bazı uygulamalarda, PicoBlaze mikrodenetleyiciye veri sağlayan devrenin verilerin başarıyla okunduğunu bilmesi gerekir. Aşağıda bir FIFO örneği bu durumu açıklar. FIFO, yalnızca PicoBlaze'in READ\_STROBE sinyali ile veriyi başarıyla aldığını onayladıktan sonra okuma işaretçisini günceller. Bu işlem, veri bütünlüğünü korurken sistem güvenilirliğini artırır.

Bu tasarım yaklaşımı, hem giriş işlemlerinin performansını optimize eder hem de zamanlama ve veri bütünlüğü açısından güvenilir çözümler sunar.

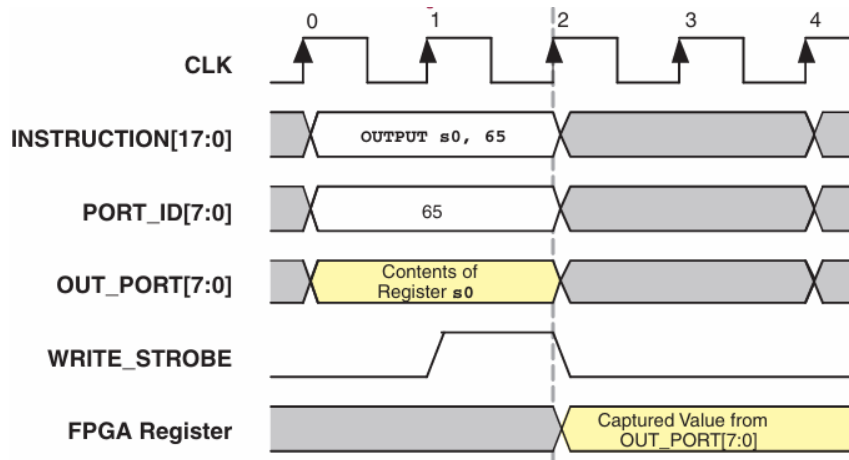


## OUTPUT Operations (Çıkış İşlemleri)

PicoBlaze mikrodnetleyicisi, bir OUTPUT işlemi sırasında 16 kayıttan herhangi birinin içeriğini OUT\_PORT çıkış bağlantı noktasına aktarır. Hedef çıkış adresi, PORT\_ID bağlantı noktasında belirtilir ve bu adres, ya bir sY kaydından ya da 8 bitlik sabit bir değerden elde edilir. Çıkış verilerinin geçerli olduğunu göstermek için WRITE\_STROBE sinyali kullanılır. Bu sinyal, FPGA'daki diğer modüllere verinin hazır olduğunu ve yakalanabileceğini bildirir.



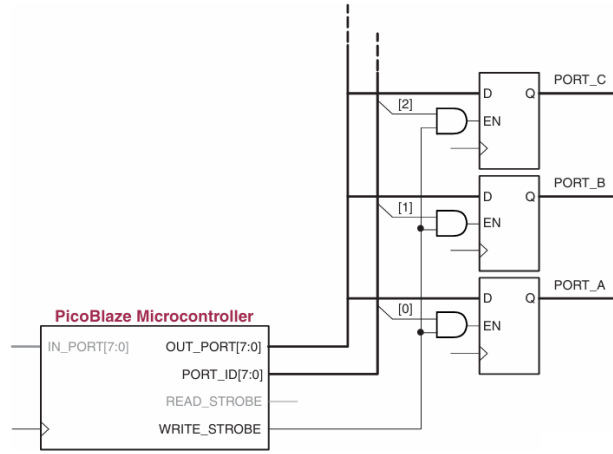
OUTPUT komutunun iki saat döngüsü vardır. İlk döngüde PORT\_ID ve OUT\_PORT değerleri atanır, ikinci döngüde ise WRITE\_STROBE sinyali yükselir ve çıkış verisinin geçerliliği onaylanır. Örneğin, bir işlem sırasında s0 kaydının içeriği, onaltılık 65 adresine yazılır. Bu sırada s0 verisi OUT\_PORT'ta, port adresi ise PORT\_ID'de görünür.



Az sayıda çıkış hedefi olan sistemlerde, adresleme mantığını basitleştirmek mümkündür. "Tek sıcak" (one-hot) adresleme gibi yöntemler, sistemin donanım maliyetini azaltırken performansını artırır. Aşağıda gösterildiği üzere, sekiz veya daha az çıkış portuna sahip tasarımlarda bu yöntem, adres çözme mantığını sadeleştirir ve PORT\_ID veri yolu

üzerindeki yükü önemli ölçüde azaltır. Bu sayede, özellikle zamanlama ve performans açısından kritik tasarımlarda yüksek verimlilik elde edilir.

Daha karmaşık tasarımlarda bile, PORT\_ID ve çıkış yollarındaki yüklerin kontrol altında tutulması sistemin saat hızını korumak ve güvenilirliği artırmak açısından kritik öneme sahiptir. Bu nedenle, tasarım sırasında adresleme ve çıkış mantığı optimize edilmelidir.



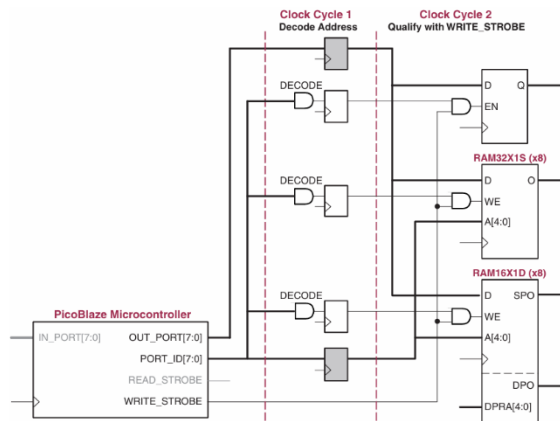
### Maksimum Performans için Boru Hattı

PicoBlaze mikrodenetleyicisi, çoğu uygulamada fazlasıyla yeterli performans sunar. Ancak, birden fazla bellek bloğuna bağlı veya çok sayıda basit bağlantı noktasına sahip tasarımlarda, sistem kaynaklarının etkili kullanımı ve maksimum performans için özel optimizasyonlar gerekebilir. Özellikle 256 adet PORT\_ID bağlantı noktasının çoğunu kullanan tasarımlarda, adres kod çözme ve yönlendirme işlemleri genel sistem tasarımını karmaşıktırabilir. Bu zorlukları aşmanın bir yöntemi, PORT\_ID kod çözme mantığının boru hattı (pipeline) düzeninde optimize edilmesidir.

### ÇIKIŞ İşlemlerinde Performans Optimizasyonu

Bir ÇIKIŞ işlemi sırasında, PORT\_ID ve OUT\_PORT bağlantı noktaları iki saat döngüsü boyunca etkin kalır. Ancak WRITE\_STROBE sinyali yalnızca ikinci döngüde yükselir ve verinin geçerliliğini onaylar. Performansı artırmak için, PORT\_ID kod çözme işlemi boru hattı yapısına dönüştürülebilir. Aşağıdaki görselde gösterildiği gibi, PORT\_ID mantığının boru hattı düzeneği ile çalıştırılması, yükleme ve yayılma sorunlarını azaltır.

Boru hattı ile PORT\_ID değerleri flip-flop'lar içinde yakalanır ve bir sonraki saat döngüsünde WRITE\_STROBE sinyaliyle nitelendirilir. Bu düzenleme, hem verinin FPGA mantığına doğru bir şekilde aktarılmasını hem de maksimum performansın korunmasını sağlar. OUT\_PORT ve PORT\_ID sinyallerindeki boru hattı kayıtları isteğe bağlıdır. Ancak, bu kayıtların kullanılması yönlendirme mesafesini azaltır ve yayılma gecikmelerini minimize ederek sistem performansını artırır.

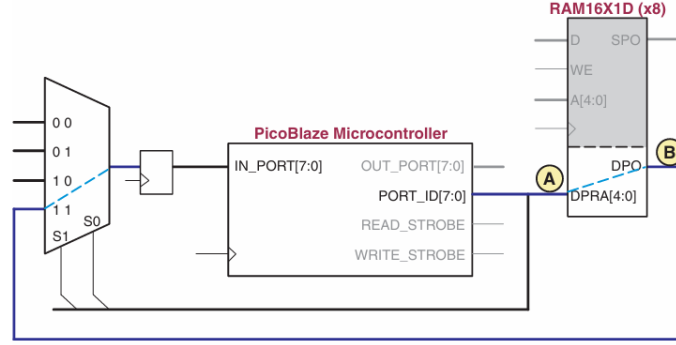


## GİRİŞ İşlemlerinde Boru Hattı Zorlukları

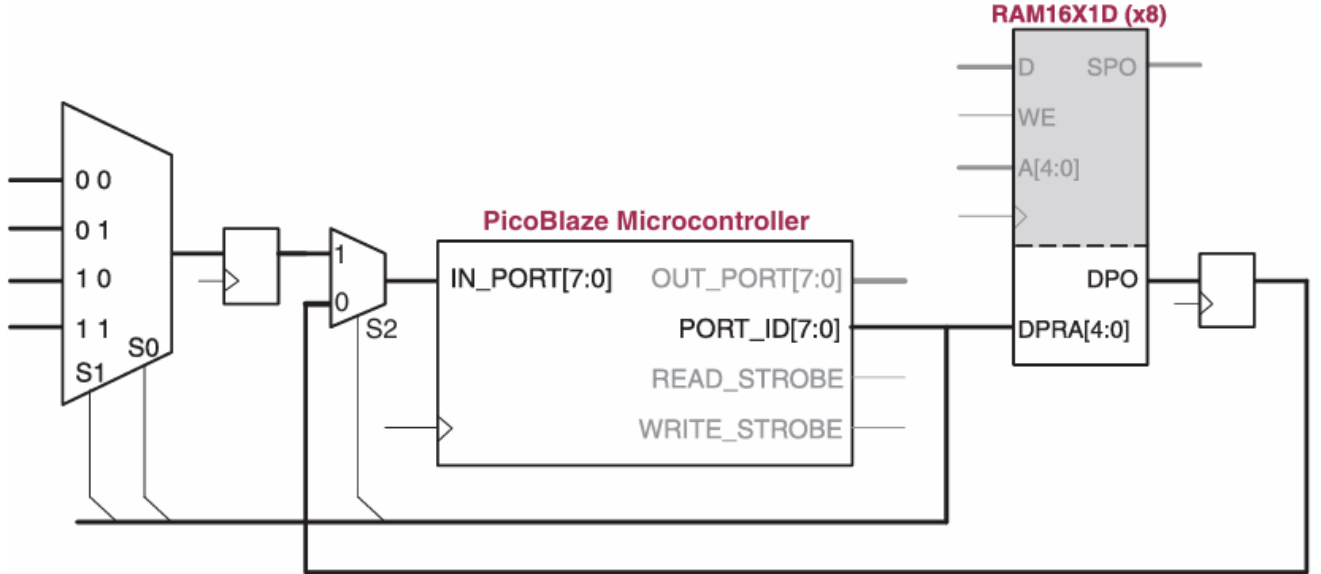
GİRİŞ işlemleri sırasında, PicoBlaze mikrodenetleyicisi veriyi FPGA mantığından talep eder ve bu veriyi alana kadar işlemi tamamlamaz. Kritik zamanlama yollarının optimize edilmesi, bu süreçte performansın artırılmasında kilit rol oynar.

Örneğin, çift bağlantı noktalı bir RAM'den veri okuyan bir tasarımda, PORT\_ID sinyali giriş adresini belirtir. Kritik yol; PORT\_ID'den RAM okuma yolu, giriş seçici çoklayıcı ve boru hattı kaydına kadar uzanır. Bu yol performans sınırlayıcı hale gelirse, boru hattı kaydı eklenerek bu sorun giderilebilir.

### Kritik Zamanlama Yollarının Değerlendirilmesi



Kritik zamanlama yollarında boru hattı kaydı eklemek, performansı artırmanın en etkili yollarından biridir. Ancak kaydın doğru konumlandırılması önemlidir. Yukarıdaki görselde gösterildiği gibi, kritik yolun hangi noktada performansı sınırladığını belirlemek gerekir. Nokta A veya Nokta B gibi boru hattı kayıtları için potansiyel yerler, tasarımın zamanlama gereksinimlerine göre dikkatle değerlendirilmelidir.



Sonuç olarak hem GİRİŞ hem de ÇIKIŞ işlemleri sırasında performansı artırmak için boru hattı tasarımı kritik öneme sahiptir. Doğru optimizasyonlar, tasarımın genel verimliliğini artırır ve PicoBlaze mikrodenetleyicisi ile daha karmaşık ve yüksek performanslı sistemlerin oluşturulmasına olanak tanır.

### Giriş (INPUT) İşlemleri ve Performans Optimizasyonu

Aşağıdaki şekile göre INPUT işlemleri sırasında okuma verilerinin, ikinci saat döngüsünün sonunda IN\_PORT bağlantı noktasında mevcut ve geçerli olması gereklidir. Giriş seçme çoklayıcısının hemen arkasında yer alan mevcut boru hattı katmanı, bu gereksinimi destekler. Ancak, boru hattı kayıtlarının yerleştirilmesi sırasında dikkatli olunması gerekir. A Noktası ya da B Noktası gibi olası yerler değerlendirilirken, yanlış bir konumlandırma ek bir saat döngüsü gecikmesine yol

açabilir ve PicoBlaze mikrodnetleyicisinin zamanlama gereksinimlerini karşılayamayabilir.

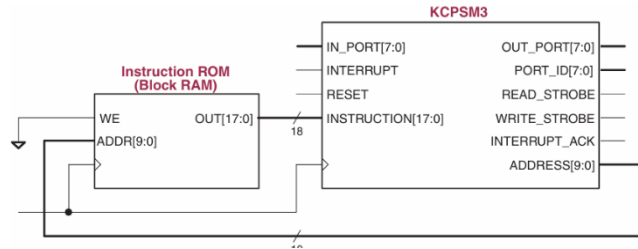
Boru hattı kaydını konumlandırmak için en uygun yer, okuma yolunu kabaca ikiye bölen B Noktasıdır. Bu nokta, giriş seçme çoklayıcı yapısında önemli bir değişiklik yapılmasını gerektirir. Şekil 6-11'de gösterildiği gibi, çoklayıcı yapısı ekstra kayıt katmanını barındıracak şekilde yeniden yapılandırılmalıdır. Bu optimizasyon, giriş seçme yolundaki yayılma ve gecikme sorunlarını azaltır ve okuma işlemlerinin zamanlama gereksinimlerini karşılamasını sağlar.

### Talimat Depolama Yapılandırmaları

PicoBlaze mikrodnetleyici, FPGA içerisinde yerleşik olarak bulunan bellek kaynaklarından kod yürütür. Mikrodnetleyicinin yapısı, iki temel alt fonksiyondan oluşur:

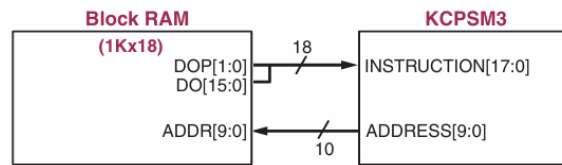
1. **KCPSM3 Modülü:** Bu modül, PicoBlaze ALU, kayıt dosyası, karalama defteri RAM'i ve diğer işlemci bileşenlerini içerir.
2. **Komut Deposu:** Genellikle bir blok RAM'den oluşan bu yapı, PicoBlaze mikrodnetleyicisinin ROM olarak kullandığı bellek alanını sağlar. Blok RAM'in yazma etkinleştirme pini (WE), düşük seviyede tutulur ve yazma işlemleri devre dışı bırakılarak sabit bir ROM yapısı elde edilir.

Bununla birlikte, PicoBlaze mikrodnetleyici, bazı uygulamalar için alternatif bellek yapılandırmalarını da destekler. Bu yapılandırmaların çoğu, Spartan-3, Spartan-6® ve Virtex-6® FPGA'larda bulunan çift bağlantı noktalı blok RAM'in sunduğu ek bağlantı noktasından faydalanır.



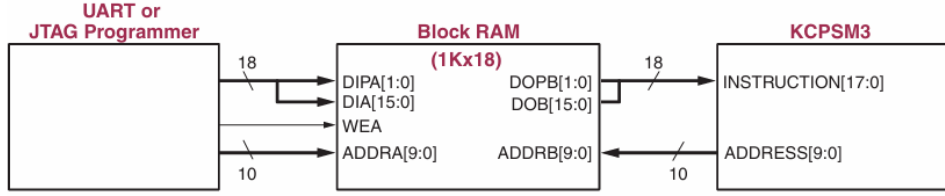
### Standart Konfigürasyon: Tek 1Kx18 Blok RAM

Çoğu uygulamada, PicoBlaze talimatları tek bir 1Kx18 boyutundaki blok RAM'de saklanır. Bu blok RAM, FPGA tasarımının bir parçası olarak yapılandırılır ve talimat deposu FPGA konfigürasyonu sırasında otomatik olarak yüklenir. Bu yapılandırma, mikrodnetleyicinin ROM işlevselliği ile uyumludur ve basit bir tasarım yaklaşımı sunar.



### UART veya JTAG Programlama Arayüzü ile Standart Yapılandırma

Blok RAM'in çift bağlantı noktası yeteneği, PicoBlaze mikrodnetleyicisinin kod deposunu tüm FPGA tasarımını yeniden derlemeye gerek kalmadan güncellemeye olanak tanır. Bu özellik, işlemcinin durdurulması ve uygulama kodunun UART veya FPGA'nın JTAG bağlantı noktası aracılığıyla yüklenmesiyle gerçekleştirilir. Bu yöntem, özellikle hızlı prototipleme ve geliştirme süreçlerinde kullanışlıdır.

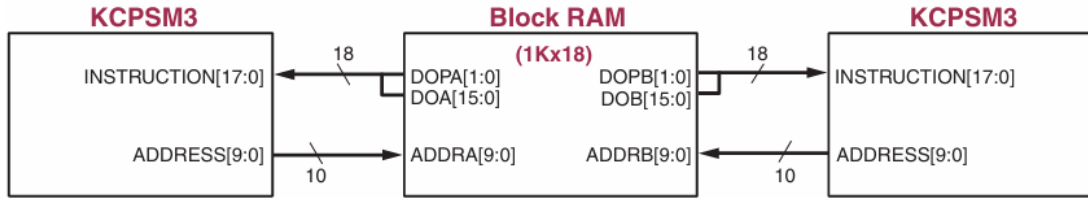


### Ortak Kod Görüntüsünü Paylaşan İki PicoBlaze Mikrodenetleyici

Şekil 7-4'te gösterildiği gibi, iki PicoBlaze mikrodenetleyici, aynı veya çoğunlukla ortak bir kod görüntüsünü depolamak için tek bir çift bağlantı noktalı blok RAM'i paylaşabilir. Bu yapılandırma, aşağıdaki avantajları sunar:

- **Bağımsız Çalışma:** Her bir mikrodenetleyici, saat girişi, G/Ç bağlantı noktaları ve kesme girişi bakımından benzersizdir ve bağımsız olarak çalışır.
- **Kaynak Optimizasyonu:** Çift bağlantı noktalı RAM kullanımı, bellek kaynaklarını verimli bir şekilde kullanır ve iki işlemci arasında kod paylaşımı sağlar.

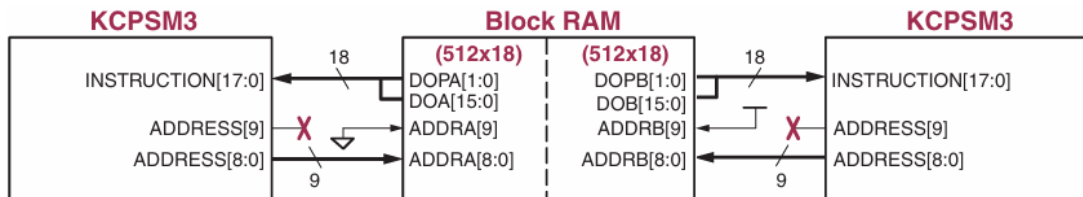
Bu yapılandırma, özellikle bellek alanının sınırlı olduğu durumlarda ve paralel işlemlerin gerektiği uygulamalarda faydalıdır.



### Blok RAM'de Ayrı Kod Görüntülerini Paylaşan İki PicoBlaze Mikrodenetleyici

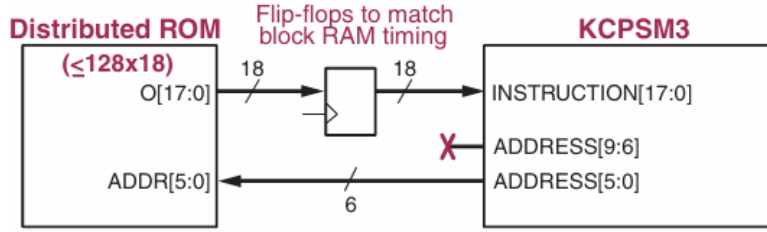
PicoBlaze mikrodenetleyicileri, tek bir çift bağlantı noktalı RAM'i paylaşarak her biri için ayrı kod alanları oluşturabilir. Bu yapılandırma, kaynakların verimli kullanımını sağlar ve her mikrodenetleyicinin bağımsız olarak çalışmasına olanak tanır. Aşağıdaki şekilde gösterildiği gibi, 1Kx18 boyutundaki blok RAM, iki eşit yarıya bölünerek her mikrodenetleyiciye 512x18 boyutunda bir komut alanı atanabilir.

- **Bağlantı Noktalarının Ayrımı:** Bir RAM bağlantı noktasının en önemli adres biti düşük seviyeye bağlanırken, diğer bağlantı noktasındaki aynı bit yüksek seviyeye bağlanır. Bu yöntem, her bir bağlantı noktasını RAM'in yarısıyla sınırlar.
- **Kesme Vektörleri:** Her iki mikrodenetleyici, RAM'in yarısını kullanmasına rağmen kesme vektörleri aynıdır. Bir kesme meydana geldiğinde, ilgili KCPSM3 bloğu, kendi RAM alanındaki son bellek konumuna (adres 1FFh) erişim sağlar.
- **Dinamik Bölümleme:** Eğer bir mikrodenetleyici diğerinden daha fazla bellek gerektiriyorsa, RAM'in üst adres bitleri uygun şekilde bağlanarak esnek bir yapılandırma sağlanabilir. Bu yapılandırma, toplam kod boyutunun 1.024x18'i aşmadığı durumlarda uygulanabilir.



## Blok RAM Yerine Dağıtılmış ROM Kullanımı

Blok RAM, PicoBlaze uygulama kodlarını depolamak için genellikle en uygun yöntemdir. Ancak tüm blok RAM'ler başka amaçlarla kullanılmışsa, FPGA'nın Yapılandırılabilir Mantık Blokları (CLB'leri) içerisinde dağıtılmış ROM kullanılabilir.



- Dağıtılmış ROM'un Avantajları ve Kısıtlamaları:** Dağıtılmış ROM, CLB içerisindeki Arama Tablolarını (LUT) küçük bir ROM olarak kullanır. Bu yöntem, 128 talimat veya daha küçük program boyutları için etkili bir seçenektir. Ancak daha büyük talimat depoları, CLB kaynaklarını hızla tüketir.
- Donanımsal Uyumluluk:** Blok RAM ile uyumluluk sağlamak adına, dağıtılmış ROM'un CLB içerisinde kayıtlı bir çıkışa sahip olması gerekir. Bu, performansın korunmasına yardımcı olur.
- Kaynak Kullanımı:** Farklı büyüklüklerdeki talimat depoları için gereken FPGA dilimlerinin miktarını gösterir. CORE Generator yazılımı, PicoBlaze assembler tarafından üretilen katsayı dosyasını kullanarak gerekli dağıtılmış ROM yapılandırmasını otomatik olarak oluşturabilir.

| Talimatlar | ROM Slices |
|------------|------------|
| ≤16        | 9          |
| ≤32        | 18         |
| ≤64        | 36         |
| ≤128       | 72         |
| ≤256       | 144        |
| ≤512       | 297        |
| ≤1024      | 594        |

Bu yapılandırmalar, FPGA içindeki kaynakları optimize etmek ve performansı artırmak için esnek çözümler sunar. Blok RAM ve dağıtılmış ROM seçenekleri, tasarım gereksinimlerine göre dikkatlice değerlendirilmeli ve uygulanmalıdır.

## Giriş Saat Frekansı ve Yürütme Performansı

PicoBlaze mikrodeneleyicisi, farklı FPGA aileleri ve hız derecelerine bağlı olarak çeşitli maksimum performans seviyelerine sahiptir. Tablo 8-1, bu performans seviyelerini özetler. Spartan-3® FPGA ailesi, düşük maliyetli uygulamalar için optimize edilmiş olup, mutlak maksimum performans gerektiren durumlar haricinde, PicoBlaze mikrodeneleyicisinin maksimum saat frekansında çalıştırılması genellikle gerekli değildir.

### Düşük Saat Frekansının Avantajları

PicoBlaze mikrodeneleyicisinin genellikle seri iletişim veya klavye izleme gibi düşük hızlı çevresel işlemleri yönettiği göz önüne alındığında, daha düşük bir saat frekansı kullanmak avantajlıdır.

- Boşta Kalma Döngülerinin Azalması:** Daha düşük saat frekansı, komut döngüleri arasındaki boşta kalma süresini azaltarak daha verimli bir çalışma sağlar.
- Güç Tüketiminin Düşürülmesi:** Saat frekansını düşürmek, sistemin toplam güç tüketimini azaltır ve özellikle enerji tasarrufu gerektiren uygulamalarda önemli bir avantaj sunar.

- **Statik Tasarım Özelliği:** PicoBlaze mikrodenetleyici tamamen statik bir tasarıma sahiptir. Bu nedenle, 0 MHz (DC) frekansında bile çalışabilir, bu da sistem tasarımında esneklik sağlar.

| FPGA Ailesi (Hız Derecesi) | Maksimum Saat Frekansı | Maksimum Yürütme Performansı |
|----------------------------|------------------------|------------------------------|
| Spartan-3 FPGA             | 125.3 MHz              | TBD                          |
| Spartan-6 FPGA             | 128 MHz                | TBD                          |
| Virtex-6 FPGA              | 165 MHz                | TBD                          |

#### Yürütme Performansını Tahmin Etme

PicoBlaze mikrodenetleyicisi, sabit bir yürütme hızı sağlayan oldukça öngörülebilir bir yapıya sahiptir:

- **Talimat Yürütme Süresi:** Tüm talimatlar her zaman iki saat döngüsü içinde yürütülür. Bu sabit yürütme hızı, özellikle gerçek zamanlı uygulamalarda büyük avantaj sağlar.
- **Programlama Kolaylığı:** Sabit bir yürütme süresi, program yürütme sürelerinin hesaplanmasını basitleştirir ve bu da zamanlama açısından hassas uygulamalarda PicoBlaze'in öne çıkmasını sağlar.

Sonuç olarak, uygulamanın gereksinimlerine göre saat frekansı ayarlanabilir. Yavaş çevresel işlemler için daha düşük bir frekans tercih edilerek enerji tasarrufu sağlanırken, kritik zamanlama gerektiren uygulamalar için performans sınırlarına ulaşılabilir. PicoBlaze'in statik tasarımı ve sabit yürütme süreleri, kullanıcıya hem esneklik hem de güvenilirlik sunar.

#### PicoBlaze ve 8085 Mikroişlemcileri Karşılaştırma Tablosu

| Özellik                            | PicoBlaze Mikrodenetleyici                          | 8085 Mikroişlemcisi                                       |
|------------------------------------|-----------------------------------------------------|-----------------------------------------------------------|
| <b>Mimarisi</b>                    | 8-bit RISC (Reduced Instruction Set Computing)      | 8-bit CISC (Complex Instruction Set Computing)            |
| <b>Üretici</b>                     | Xilinx                                              | Intel                                                     |
| <b>Üretim Yılı</b>                 | İlk sürüm: 2000'ler                                 | İlk sürüm: 1976                                           |
| <b>Kullanım Alanı</b>              | FPGA içinde gömülü uygulamalar                      | Genel amaçlı mikroişlemci uygulamaları                    |
| <b>Saat Hızı</b>                   | Maksimum 200 MHz                                    | Maksimum 3 MHz                                            |
| <b>Komut Seti Yapısı</b>           | 64 adet temel komut                                 | 74 adet temel komut                                       |
| <b>Komut Türleri</b>               | Veri işleme, kontrol akışı, giriş/çıkış işlemleri   | Aritmetik, mantık, veri transferi, kontrol akışı          |
| <b>Komut Çevrim Sayısı</b>         | Her komut sabit olarak 2 saat döngüsünde yürütülür. | 4 ila 18 saat döngüsü, komut türüne bağlı olarak değişir. |
| <b>Talimat Seti Basitliği</b>      | Minimal ve optimize edilmiş.                        | Daha geniş ve kompleks                                    |
| <b>Hafıza Mimarisi</b>             | Dahili ROM/Blok RAM                                 | Harici RAM ve ROM bağlantısı                              |
| <b>Bellek Adresleme Kapasitesi</b> | 1 KB komut deposu (1024 x 18 bit)                   | 64 KB (16-bit adres yolu)                                 |
| <b>Veri Yolu Genişliği</b>         | 8 bit                                               | 8 bit                                                     |
| <b>Adres Yolu Genişliği</b>        | 10 bit                                              | 16 bit                                                    |
| <b>Giriş/Çıkış Portları</b>        | 256 adreslenebilir I/O portu                        | 256 adreslenebilir I/O portu                              |
| <b>Kesme Mekanizması</b>           | 1 öncelikli kesme girişi                            | 5 donanım kesmesi (TRAP, RST7.5, RST6.5, RST5.5, INTR)    |
| <b>Kesme Vektörleri</b>            | Program kodunda saklanır.                           | Sabit kesme vektörleri (Örneğin, TRAP için 0024H adresi)  |

|                                      |                                                            |                                                              |
|--------------------------------------|------------------------------------------------------------|--------------------------------------------------------------|
| <b>Güç Tüketimi</b>                  | Saat frekansı azaltılarak düşük güç tüketimi sağlanabilir. | Sabit ve nispeten yüksek güç tüketimi                        |
| <b>Çalışma Frekansı Esnekliği</b>    | 0 MHz (statik çalışma desteği)                             | Statik çalışma desteklenmez.                                 |
| <b>Saat Döngüsü Performansı</b>      | Sabit komut yürütme süresi                                 | Çevrim sayısı komuta göre değişir.                           |
| <b>Çevresel İşlem Desteği</b>        | Giriş/çıkışlar FPGA üzerinden kolayca yapılandırılır.      | Harici çevresel cihazlarla uyum sağlamak için daha karmaşık. |
| <b>Harici Bileşen Gereksinimi</b>    | Yok (tüm donanım FPGA içine gömülüdür)                     | Harici RAM, ROM ve I/O cihazları gerekir.                    |
| <b>Gömülü Bellek Kullanımı</b>       | FPGA blok RAM'leri                                         | Harici ROM ve RAM                                            |
| <b>Yazılım Araçları</b>              | PicoBlaze assembler, Xilinx araçları                       | 8085 assembler ve üçüncü parti geliştirme araçları           |
| <b>Talimat Depolama</b>              | FPGA'daki Blok RAM veya dağıtılmış ROM                     | Harici ROM üzerinde depolama                                 |
| <b>Programlama Yöntemi</b>           | Kod FPGA tasarımının bir parçası olarak oluşturulur.       | Kod, bağımsız olarak ROM içine yazılır.                      |
| <b>Çift Mikrodenetleyici Desteği</b> | Blok RAM'i paylaşan iki bağımsız PicoBlaze çalışabilir.    | Tek bir işlemci                                              |
| <b>Kod Güncelleme</b>                | UART veya JTAG üzerinden canlı güncelleme                  | ROM'u yeniden programlama gerekir.                           |
| <b>Gerçek Zamanlı Performans</b>     | Sabit yürütme süresiyle tahmin edilebilir.                 | Gerçek zamanlı performans öngörüsü zordur.                   |
| <b>Kod Verimliliği</b>               | Optimize edilmiş ve kompakt                                | Daha geniş, düşük verimli kodlama                            |
| <b>Çevresel İşlem Yönetimi</b>       | Seri iletişim, tuş takımı okuma gibi yavaş işlemlere uygun | Daha genel çevresel cihazlarla yönetim sağlar.               |
| <b>Kullanım Kolaylığı</b>            | FPGA kullanıcıları için optimize edilmiştir.               | Harici donanım ve bağlantılar gerektirir.                    |
| <b>Özelleştirilebilirlik</b>         | Kullanıcı tarafından FPGA tasarımıyla özelleştirilebilir.  | Sabit bir tasarıma sahiptir, özelleştirilemez.               |
| <b>Evrensel Uyumluluk</b>            | Sadece Xilinx FPGA ailesi ile uyumlu                       | Genel olarak farklı platformlarla uyumlu                     |
| <b>Geliştirme Zorluğu</b>            | FPGA sistemleri için kolay entegre                         | Daha fazla donanım ve bağlantı gerektirir.                   |
| <b>Üretim Teknolojisi</b>            | FPGA içindeki bloklar olarak uygulanır.                    | Silikon tabanlı bağımsız çip                                 |
| <b>Statik/Dinamik Çalışma</b>        | Statik tasarım, DC (0 MHz) destekler.                      | Dinamik tasarım, DC desteklemez.                             |
| <b>Performans Hedefi</b>             | Gömülü ve düşük maliyetli uygulamalar.                     | Genel amaçlı işlemci performansı                             |

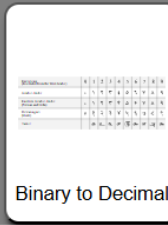
# PicoBlaze Simulator

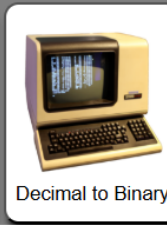
Front-end made by [Teo Samaržija](#) and [Agustin Izaguirre](#).

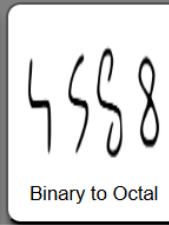
Back-end made by [Abidin Durdu](#).

```
1. ORG 0 ; Başlangıç adresi
2. START:
3. LOAD s0, 03 ; İlk sayıyı yükle (3)
4. LOAD s1, 07 ; İkinci sayıyı yükle (7)
5. LOAD s2, 00 ; Sonucu sıfırla
6. ADD s2, s0 ; s2 = s2 + s0
7. ADD s2, s1 ; s2 = s2 + s1 (toplama işle
8. OUTPUT s2, 01 ; Sonucu çıkış portuna yazdı
9. LOOP:
10. JUMP LOOP ; Programı burada sonsuz dön
; Programın sonu
```

Example s:

  
Binary to Decimal

  
Decimal to Binary

  
Binary to Octal

  
Fibonacci Sequence

Highlight 



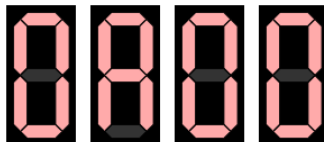
Assemble

Please click the ["Assemble" button](#) before proceeding, or else you will be simulating PicoBlaze that constantly loops `load s0, s0` (which is `00000` in hexadecimal).



Clicking the button above will download the hexadecimal (.hex) file, in the same format that the Xilinx PicoBlaze Assembler outputs. You need to convert it to a binary file before uploading it to PicoBlaze. I am sorry that this program is not outputting binary files, but there is no obvious way to do it since PicoBlaze instructions are 18-bit and the smallest addressable memory unit in JavaScript is byte (8 bits), and 18 bits is not divisible by bytes.

| Machine Code |         |           |      |
|--------------|---------|-----------|------|
| PC           | Address | Directive | Line |
|              | 000     | 01003     | 3    |
|              | 001     | 01107     | 4    |
|              | 002     | 01200     | 5    |
|              | 003     | 10200     | 6    |
|              | 004     | 10210     | 7    |
|              | 005     | 2d201     | 8    |
| →            | 006     | 22006     | 10   |
|              | 007     | 00000     | 0    |



**Enable UART**

| Registers  | s0 | s1 | s2 | s3 | s4 | s5 | s6 | s7 | s8 | s9 | sa | sb | sc | sd | se | sf | PC  |
|------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|
| REG BANK A | 03 | 07 | 0a | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 006 |
| REG BANK B | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00  |

| Flags      | Zero (Z) | Carry (C) | Interrupt Enabled (IE) |
|------------|----------|-----------|------------------------|
| REG BANK A | 0        | 0         | 1                      |
| REG BANK B | 0        | 0         |                        |

**NOTE:** I have good reasons to think the emulation of flags is unrealistic, especially when it comes to REGBANKS.

| Address | Input | Output | Memory |
|---------|-------|--------|--------|
| 00      | 00    | 00     | 00     |
| 01      | 00    | 0a     | 00     |
| 02      | 00    | 00     | 00     |
| 03      | 00    | 00     | 00     |
| 04      | 00    | 00     | 00     |
| 05      | 00    | 00     | 00     |
| 06      | 00    | 00     | 00     |

### Picoblaze Kod

```

ORG 0      ; Başlangıç adresi

START:

LOAD s0, 03    ; İlk sayıyı yükle (3)
LOAD s1, 07    ; İkinci sayıyı yükle (7)
LOAD s2, 00    ; Sonucu sıfırla
ADD s2, s0     ; s2 = s2 + s0
ADD s2, s1     ; s2 = s2 + s1 (toplama işlemi 3+7)
OUTPUT s2, 01  ; Sonucu çıkış portuna yazdır

LOOP:

JUMP LOOP      ; Programı burada sonsuz döngüde tut
; Programın sonu

```

## PicoBlaze Komut Seti

|                               | Açıklama                                                                                                                                | Fonksiyon                                                                                                                                               | ZERO | CARRY |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------|-------|
| ADD sX, kk                    | Değişmez kk ile sX kaydını ekleyin.                                                                                                     | $sX \leftarrow sX + kk$                                                                                                                                 | -    | -     |
| ADD sX, sY                    | sY kaydı ile sX kaydını ekleyin.                                                                                                        | $sX \leftarrow sX + sY$                                                                                                                                 | -    | -     |
| ADDCY sX, kk (ADDC)           | CARRY biti ile kk değişmez değeri ile sX kaydını ekleyin.                                                                               | $sX \leftarrow sX + kk + CARRY$                                                                                                                         | -    | -     |
| ADDCY sX, sY (ADDC)           | CARRY biti ile sY kaydını sX kaydına ekleyin.                                                                                           | $sX \leftarrow sX + sY + CARRY$                                                                                                                         | -    | -     |
| AND sX, kk                    | Bitsel VE ile sX'i kk değişmez değeri ile kaydedin.                                                                                     | $sX \leftarrow sX \text{ AND } kk$                                                                                                                      | -    | -     |
| AND sX, sY                    | Bitsel VE ile sX'i sY kaydı ile kaydedin.                                                                                               | $sX \leftarrow sX \text{ AND } sY$                                                                                                                      | -    | -     |
| CALL aaa                      | Aaa'da koşulsuz olarak alt yordamı çağırın.                                                                                             | $TOS \leftarrow PC$<br>$PC \leftarrow aaa$                                                                                                              | -    | -     |
| CALL C, aaa                   | CARRY bayrağı ayarlandıysa, aaa'da alt yordamı çağırın.                                                                                 | If CARRY=1, { $TOS \leftarrow PC$ ,<br>$PC \leftarrow aaa$ }                                                                                            | -    | -     |
| CALL NC, aaa                  | CARRY bayrağı ayarlanmamışsa, aaa'da alt yordamı çağırın.                                                                               | If CARRY=0, { $TOS \leftarrow PC$ ,<br>$PC \leftarrow aaa$ }                                                                                            | -    | -     |
| CALL NZ, aaa                  | SIFIR bayrağı ayarlanmadıysa, aaa'da alt yordamı çağırın.                                                                               | If ZERO=0, { $TOS \leftarrow PC$ ,<br>$PC \leftarrow aaa$ }                                                                                             | -    | -     |
| CALL Z, aaa                   | SIFIR bayrağı ayarlanırsa, aaa'da alt yordamı çağırın.                                                                                  | If ZERO=1, { $TOS \leftarrow PC$ ,<br>$PC \leftarrow aaa$ }                                                                                             | -    | -     |
| COMPARE sX, kk (COMP)         | Register sX'i literal kk ile karşılaştırın. CARRY ve ZERO bayraklarını uygun şekilde ayarlayın. Kayıtlar etkilenmez.                    | If $sX=kk$ , ZERO $\leftarrow 1$<br>If $sX < kk$ , CARRY $\leftarrow 1$                                                                                 | -    | -     |
| COMPARE sX, sY (COMP)         | sX kaydını sY kaydı ile karşılaştırın. CARRY ve ZERO bayraklarını uygun şekilde ayarlayın. Kayıtlar etkilenmez.                         | If $sX=sY$ , ZERO $\leftarrow 1$<br>If $sX < sY$ , CARRY $\leftarrow 1$                                                                                 | -    | -     |
| DISABLE INTERRUPT (DINT)      | Kesme girişini devre dışı bırakın.                                                                                                      | INTERRUPT_ENABLE $\leftarrow 0$                                                                                                                         | -    | -     |
| ENABLE INTERRUPT (EINT)       | Kesme girişini etkinleştir                                                                                                              | INTERRUPT_ENABLE $\leftarrow 1$                                                                                                                         | -    | -     |
| Interrupt Event               | Asenkron kesme girişi. Bayrakları ve bilgisayarı koruyun. INTERRUPT_ENABLE işaretini kaldırın. 3FF adresindeki kesme vektörüne atlayın. | Preserved ZERO $\leftarrow$ ZERO<br>Preserved CARRY $\leftarrow$ CARRY<br>INTERRUPT_ENABLE $\leftarrow 0$<br>$TOS \leftarrow PC$<br>$PC \leftarrow 3FF$ | -    | -     |
| FETCH sX, (sY) (FETCH sX, sY) | sY kaydı tarafından işaret edilen karalama defteri RAM konumunu sX kaydına okuyun                                                       | $sX \leftarrow RAM[(sY)]$                                                                                                                               | -    | -     |
| FETCH sX, ss                  | Scratchpad RAM konumunu ss sX'i register'a okuyun                                                                                       | $sX \leftarrow RAM[ss]$                                                                                                                                 | -    | -     |
| INPUT sX, (sY) (IN sX, sY)    | sY kaydı tarafından sX kaydına işaret edilen                                                                                            | PORT_ID $\leftarrow sY$<br>$sX \leftarrow IN\_PORT$                                                                                                     | -    | -     |

|                                 |                                                                                          |                                                                                      |   |   |
|---------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---|---|
|                                 | giriş bağlantı noktası konumundaki değeri okuyun                                         |                                                                                      |   |   |
| INPUT sX, pp (IN)               | sX kaydına giriş portu konumu pp üzerindeki değeri okuyun                                | PORT_ID ← pp<br>sX ← IN_PORT                                                         | - | - |
| JUMP aaa                        | Koşulsuz olarak aaa'ya atlayın                                                           | PC ← aaa                                                                             | - | - |
| JUMP C, aaa                     | CARRY bayrağı ayarlandıysa, aaa'ya atlayın                                               | If CARRY=1, PC ← aaa                                                                 | - | - |
| JUMP NC, aaa                    | CARRY bayrağı ayarlanmamışsa, aaa'ya atlayın                                             | If CARRY=0, PC ← aaa                                                                 | - | - |
| JUMP NZ, aaa                    | SIFIR bayrağı ayarlanmadıysa, aaa'ya atlayın                                             | If ZERO=0, PC ← aaa                                                                  | - | - |
| JUMP Z, aaa                     | SIFIR bayrağı ayarlandıysa, aaa'ya atlayın                                               | If ZERO=1, PC ← aaa                                                                  | - | - |
| LOAD sX, kk                     | sX kaydını kk harfiyle yükle                                                             | sX ← kk                                                                              | - | - |
| LOAD sX, sY                     | sX kaydını sY kaydıyla yükle                                                             | sX ← sY                                                                              | - | - |
| OR sX, kk                       | Bitsel VEYA sX'i kk değişmez değeri ile kaydedin                                         | sX ← sX OR kk                                                                        | ? | 0 |
| OR sX, sY                       | Bitsel VEYA sX'i kk değişmez değeri ile kaydedin                                         | sX ← sX OR sY                                                                        | ? | 0 |
| OUTPUT sX, (sY)<br>(OUT sX, sY) | Register sX'i register sY tarafından işaret edilen çıkış bağlantı noktası konumuna yazın | PORT_ID ← sY<br>OUT_PORT ← sX                                                        | - | - |
| OUTPUT sX, pp (OUT sX, pp)      | Çıktı bağlantı noktası konumuna pp kayıt sX yazın                                        | PORT_ID ← pp<br>OUT_PORT ← sX                                                        | - | - |
| RETURN (RET)                    | Koşulsuz olarak alt rutinden dön                                                         | PC ← TOS+1                                                                           | - | - |
| RETURN C (RET C)                | CARRY bayrağı ayarlanmışsa, alt yordamdan dönün                                          | If CARRY=1, PC ← TOS+1                                                               | - | - |
| RETURN NC (RET NC)              | CARRY bayrağı ayarlanmamışsa, alt yordamdan dönün                                        | If CARRY=0, PC ← TOS+1                                                               | - | - |
| RETURN NZ (RET NZ)              | SIFIR bayrağı ayarlanmamışsa, alt yordamdan dönün                                        | If ZERO=0, PC ← TOS+1                                                                | - | - |
| RETURN Z (RET Z)                | SIFIR bayrağı ayarlanırsa, alt yordamdan dönün                                           | If ZERO=1, PC ← TOS+1                                                                | - | - |
| RETURNI DISABLE (RETI DISABLE)  | Kesinti hizmeti rutininden dönün. Kesme devre dışı kalır.                                | PC ← TOS<br>ZERO ← Preserved ZERO<br>CARRY ← Preserved CARRY<br>INTERRUPT ENABLE ← 0 | ? | ? |
| RETURNI ENABLE (RETI ENABLE)    | Kesinti hizmeti rutininden dönün. Kesmeyi yeniden etkinleştirin.                         | PC ← TOS<br>ZERO ← Preserved ZERO<br>CARRY ← Preserved CARRY<br>INTERRUPT ENABLE ← 1 | ? | ? |
| RL sX                           | Kaydı sX sola döndür                                                                     | sX ← {sX[6:0], sX[7]}<br>CARRY ← sX[7]                                               | ? | ? |
| RR sX                           | Kaydı sX'i sağa döndür                                                                   | sX ← {sX[0], sX[7:1]}                                                                | ? | ? |

|                                  |                                                                                                                    |                                                                                            |   |   |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|---|---|
|                                  |                                                                                                                    | $CARRY \leftarrow sX[0]$                                                                   |   |   |
| SL0 sX                           | Kaydırma kaydı sX sola, sıfır dolgu                                                                                | $sX \leftarrow \{sX[6:0], 0\}$<br>$CARRY \leftarrow sX[7]$                                 | ? | ? |
| SL1 sX                           | Kaydırma kaydı sX sola, bir dolgu                                                                                  | $sX \leftarrow \{sX[6:0], 1\}$<br>$CARRY \leftarrow sX[7]$                                 | 0 | ? |
| SLA sX                           | CARRY dahil olmak üzere tüm bitlerde sX kaydırma kaydı                                                             | $sX \leftarrow \{sX[6:0], CARRY\}$<br>$CARRY \leftarrow sX[7]$                             | ? | ? |
| SLX sX                           | Shift register sX sola. Bit sX[0] etkilenmez.                                                                      | $sX \leftarrow \{sX[6:0], sX[0]\}$<br>$CARRY \leftarrow sX[7]$                             | ? | ? |
| SR0 sX                           | Shift register sX sağ, sıfır dolgu                                                                                 | $sX \leftarrow \{0, sX[7:1]\}$<br>$CARRY \leftarrow sX[0]$                                 | ? | ? |
| SR1 sX                           | Shift register sX sağ, bir dolgu                                                                                   | $sX \leftarrow \{1, sX[7:1]\}$<br>$CARRY \leftarrow sX[0]$                                 | 0 | ? |
| SRA sX                           | CARRY dahil olmak üzere sX'i tüm bitlerde sağa kaydırın                                                            | $sX \leftarrow \{CARRY, sX[7:1]\}$<br>$CARRY \leftarrow sX[0]$                             | ? | ? |
| SRX sX                           | Aritmetik kaydırma kaydı sX sağ. İşaret sX'i genişletir. Bit sX[7] Etkilenmez                                      | $sX \leftarrow \{sX[7], sX[7:1]\}$<br>$CARRY \leftarrow sX[0]$                             | ? | ? |
| STORE sX, (sY)<br>(STORE sX, sY) | sX kaydını sY kaydı tarafından işaret edilen karalama defteri RAM konumuna yazın                                   | $RAM[(sY)] \leftarrow sX$                                                                  | - | - |
| STORE sX, ss                     | Scratchpad RAM konumuna ss kaydı yaz                                                                               | $RAM[ss] \leftarrow sX$                                                                    | - | - |
| SUB sX, kk                       | Kaydeden kk değişmezini sX kaydından çıkarın                                                                       | $sX \leftarrow sX - kk$                                                                    | ? | ? |
| SUB sX, sY                       | sY kaydını sX kaydından çıkarın                                                                                    | $sX \leftarrow sX - sY$                                                                    | ? | ? |
| SUBCY sX, kk<br>(SUBC)           | CARRY ile sX kaydından kk değişmezini çıkarın (ödünç alın)                                                         | $sX \leftarrow sX - kk - CARRY$                                                            | ? | ? |
| SUBCY sX, sY<br>(SUBC)           | CARRY ile sX kaydından sY kaydını çıkarın (ödünç alın)                                                             | $sX \leftarrow sX - sY - CARRY$                                                            | ? | ? |
| TEST sX, kk                      | CARRY ile sX kaydından sY kaydını çıkarın (ödünç alın)                                                             | If (sX AND kk) = 0,<br>ZERO $\leftarrow$ 1<br>CARRY $\leftarrow$ odd parity of (sX AND kk) | ? | ? |
| TEST sX, sY                      | sX yazmaçındaki bitleri sX yazmaçına karşı test edin. CARRY ve ZERO bayraklarını güncelleyin. Kayıtlar etkilenmez. | If (sX AND sY) = 0,<br>ZERO $\leftarrow$ 1<br>CARRY $\leftarrow$ odd parity of (sX AND kk) | ? | ? |
| XOR sX, kk                       | Bitsel XOR kaydı sX'i kk değişmez değeri ile kaydeder                                                              | $sX \leftarrow sX \text{ XOR } kk$                                                         | ? | 0 |
| XOR sX, sY                       | Bitsel XOR kaydı sX kaydı ile sY kaydı                                                                             | $sX \leftarrow sX \text{ XOR } sY$                                                         | ? | 0 |

## KAYNAKÇA

<chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://web.itu.edu.tr/yalcinmust/EHB432/picoblaze.pdf>